

Datové struktury a algoritmy

Přednáška 10, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Administrativa, zopakování klíčových dat

- 7.12.
 - Místo přednášky je 90-ti minutový povinný on-line opakovací test. Fyzická přednáška v P3 se nekoná!
 - Možnost individuálních konzultací v čase přednášky v 3N021
- 14.12.
 - Povinné prezentace témat SP v čase přednášky
- 21.12.
 - Místo přednášky lze absolvovat zkouškový předtermín
 - Podrobnosti viz Instrukce v sekci Moodle/Závěrečné aktivity
 - Po dohodě do 19.12. je možnost konzultací v čase přednášky

Dnešní témata

- Prezentace
 - Vyhledávání v BVS
 - ADT zásobník a jeho implementace
 - Knihovna STL, obecná charakteristika (2)
 - Kontejnery množina a mapa v STL
- Hašovací tabulka, dokončení
- Bonifikovaný kvíz: Vyhledávací struktury, hašovací tab.
- Základní ADT a způsoby jejich implementace
 - Zásobník (Stack), fronta (Queue) , seznam (List), pole (Array, Vector, Matrix, ...), množina (Set, Bag), mapa (Dictionary, Map, Table)
- STL knihovna, souhrnné informace

Příklad úplného návrhu a realizace ADT List

1. Signatura/reprezentace

<code>init: -> List</code>	prázdný seznam
<code>insert(_, _): List, T -> List</code>	vlož před kurzor
<code>remove(_): List -> List</code>	odeber
<code>removePrev(_): List -> List</code>	odeber před kurzorem
<code>toNext(_): List -> List</code>	kurzor na další
<code>toPrev(_): List -> List</code>	kurzor na předchozí
<code>toBegin(_): List -> List</code>	kurzor na začátek
<code>toEnd(_): List -> List</code>	kurzor na konec
<code>empty(_): List -> bool</code>	je seznam prázdný?
<code>atBegin(_): List -> bool</code>	je kurzor na začátku?
<code>atEnd(_): List -> bool</code>	je kurzor na konci?
<code>read(_): List -> T</code>	hodnota na kurzoru

2. Obecná implementační struktura

- Např. obousměrný spojový seznam



3. Konkrétní způsob SW realizace

- Vlastní třída *List* s konkrétní soukromou a veřejnou částí včetně definice jednotlivých metod

```
class List {
public:
    List();
    ~List();
    void insert(T); // vloz pred kurzor
    void remove(); // odstran prvek na kurzoru
    void removePrev(); // odstran prvek pred kurzorem
    void toNext(); // kurzor na dalsi prvek
    void toPrev(); // kurzor na predchozi prvek
    void toBegin(); // kurzor na zacatek
    void toEnd(); // kurzor na konec
    bool empty(); // je seznam prazdny?
    bool atBegin(); // je kurzor na zacatku?
    bool atEnd(); // je kurzor na konci?
    bool read(T&); // cteni z pozice kurzoru
    friend ostream& operator<<(ostream&, const List&);
};

private:
struct Node {
    T val;
    Node *next, *prev;
    Node(T x, Node *n) { // n nesmi byt NULL
        val = x; next = n; prev = n->prev;
    }
    Node() {next = NULL; prev = NULL;}
};
Node *begin;
Node *tail;
Node *mark;
List(const List &);
List& operator=(const List&);
};
```

Implementace STL kontejnerů

- **Sekvenční kontejnery**

- array (statické pole)
- vector (dynamické pole)
- deque (dynamické pole SS)
- forward_list (SS)
- list (SS)

- **Kontejnerové adaptéry**

- stack (deque nebo vector)
- queue (deque nebo list)
- priority_queue (binární halda)

- **Asociativní kontejnery**

- set
- multiset
- map
- multimap

(vyvážený BVS)

- **Neseřazené asociativní kontejnery**

- unordered_set
- unordered_multiset
- unordered_map
- unordered_multimap

(hašovací tabulka)

Abstraktní datové typy

- Specifikace (abstraktního) datového typu
- Implementace datového typu
- Sekvenční kontejnery – různě organizované sekvence
 - Zásobník (Stack)
 - Fronta (Queue)
 - Seznam (List)
 - Pole (Array, Vector, Matrix, ...) umožňuje náhodný přístup k prvkům
- Asociativní kontejnery – pro vkládání, odebírání a vyhledávání je důležitý klíč prvku
 - Množina (Set)
 - Tabulka (Table, Map)

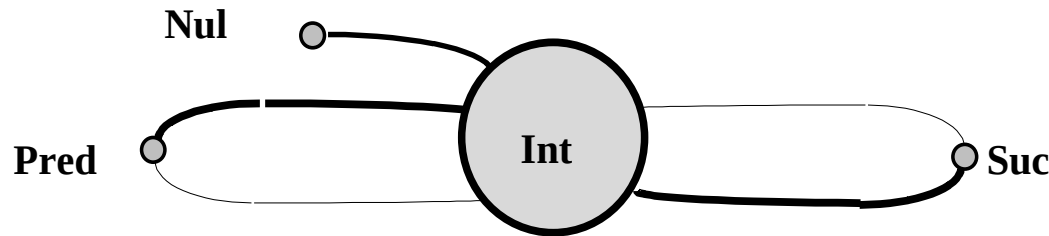
Formální specifikace ADT

Signatura ADT - syntaxe operací

Označení druhů (množiny hodnot)

Operace - vymezením druhů argumentů operací a výsledků operací

Příklad: datový typ Int



`Nul: -> Int`

`Pred(_): Int -> Int`

`Suc(_): Int -> Int`

vložení nuly

nalezení následníka

nalezení předchůdce

Použití operací:

`Nul;           ⇒ 0`

`Pred(Nul);   ⇒ -1`

`Suc(Nul);     ⇒ 1`

Formální specifikace ADT

- **Axiomatická specifikace sémantiky operací**

Axiomy jsou rovnosti výrazů

Příklad: axiomy ADT Int

$$r_1 : \text{Pred}(\text{Suc}(x)) = x$$

$$r_2 : \text{Suc}(\text{Pred}(x)) = x$$

Význam axiomů: zjednodušování výrazů

$$\text{Pred}(\text{Suc}(\text{Nul})) = \text{Nul}$$

$$\text{Suc}(\text{Pred}(\text{Nul})) = \text{Nul}$$

Programování na základě algebraických specifikací:

dokazování rovnosti výrazů zapsaných podle dané specifikace.

Předpoklad: máme k dispozici dokazovací systém, který je schopen zpracovávat symbolické výrazy

Implementace ADT

- **Neformální specifikace sémantiky operací:**
 1. Slovní popis sémantiky operací
 2. Algoritmický zápis operací

Implementace ADT

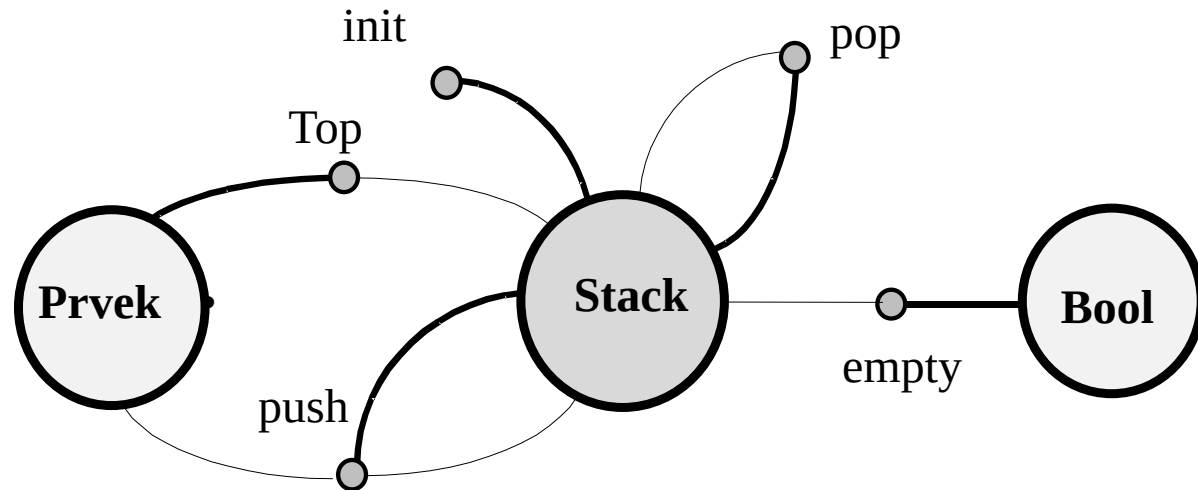
- **Výběr vhodného programovacího jazyka**
problémově orientované programovací jazyky
univerzální programovací jazyky (C++)
standardní knihovny (STL)
- **Návrh reprezentace druhů implementovaného datového typu**
základní datové struktury pole, spojový seznam a (binární) strom.
Kritéria: dynamické změny, operační a paměťová složitost
- **Návrh algoritmů operací**
realizujeme pomocí třídy,
větší použitelnost má realizace pomocí šablony třídy
- **Ověření implementace (ladění)**
ověření platnosti axiomů
- **Posouzení kvality implementace**
operační a paměťová složitost operací

Kontejnery

- **Kontejner** (kolekce) je ADT na organizované skladování prvků podle určitých pravidel
- **Sekvenční kontejnery** – různě organizované sekvence
 - Zásobník (Stack)
 - Fronta (Queue)
 - Seznam (List)
 - Pole (Array, Vector, Matrix, ...) umožňuje náhodný přístup k prvkům
- **Asociativní kontejnery** – pro vkládání, odebírání a vyhledávání je důležitý klíč prvku
 - Množina (Set)
 - Tabulka (Table, Map)
- V C++ realizujeme ADT kontejneru buď mocí **třídy** s daným typem prvků, nebo pomocí **šablony třídy**, kde typ prvků je parametrem šablony

ADT Zásobník

- Kontejner, kde vkládání a odebírání prvků probíhá na stejném konci zvaném vrchol zásobníku (LIFO)
- Signatura zásobníku



`init: → stack`

`push(_, _): prvek, stack → stack`

`pop(_): stack → stack`

`empty(_): stack → bool`

`top(_): stack → prvek`

ADT Zásobník

- Specifikace zásobníku v C++ : třída **Stack**

```
class Stack {  
public:  
    Stack() ;  
    bool empty() const;  
    void push(T x) ;  
    T top() const;  
    void pop() ;  
private:  
    ...  
};
```

- Poznámky:
 - operace *top* a *pop* bývají často spojeny do jediné:
`T pop() ;`
 - jestliže typ hodnoty prvků *T* je strukturovaný, je lépe metodě *push* předávat parametr pomocí reference:
`void push(const T& x) ;`

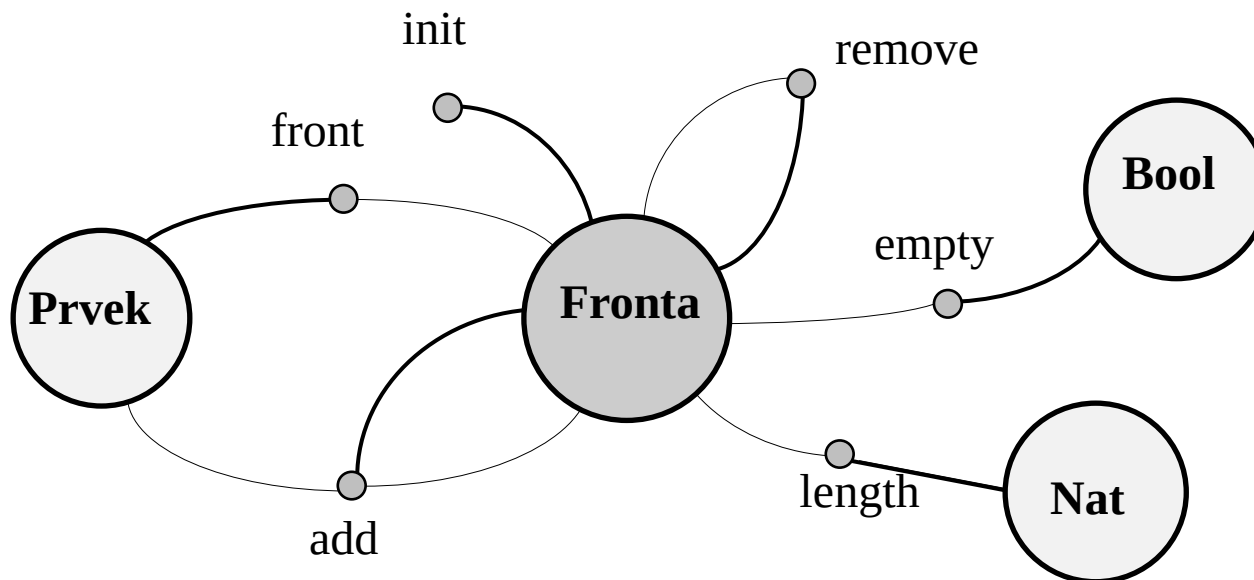
ADT Zásobník

- Implementace zásobníku:
 - pomocí **pole**, jehož počet prvků je dán konstantou (počet prvků zásobníku je staticky omezen)
 - pomocí **dynamicky alokovaného pole**, jehož počet prvků je dán parametrem konstruktoru (počet prvků zásobníku je dynamicky omezen)
 - pomocí **dynamicky alokovaného a rozšiřitelného pole**
 - pomocí jednosměrného **spojového seznamu**
- Složitost všech operací je konstantní.

Výjimkou je vkládání do zásobníku realizovaného pomocí rozšiřitelného pole: pokud se pole rozšiřuje na dvojnásobnou délku, je složitost vkládání lineární.

ADT Fronta

- Kontejner, kde se vkládá na konec a odebírá z čela (FIFO)
- Sinatura datového typu Fronta



ADT Fronta

Specifikace fronty v C++ : třída **Queue**

```
class Queue {  
public:  
    Queue() ;  
    bool empty() const;  
    void add(T x) ;  
    T front() const;  
    void remove() ;  
private:  
    ...  
};
```

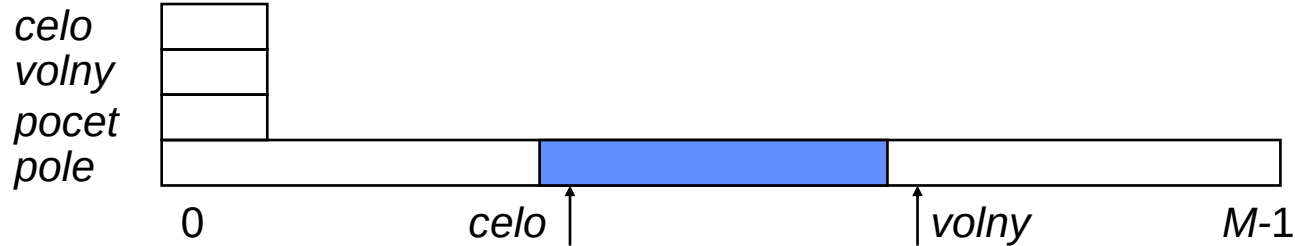
- Poznámky:
 - operace *front* a *remove* bývají často spojeny do jediné:
`T remove() ;`
 - jestliže typ hodnoty prvků *T* je strukturovaný, je lépe metodě *add* předávat parametr pomocí reference:
`void add(const T& x) ;`

ADT Fronta

- Implementace fronty:
 - pomocí **pole**, jehož počet prvků je dán konstantou (počet prvků fronty je staticky omezen)
 - pomocí **dynamicky alokovaného pole**, jehož počet prvků je dán parametrem konstruktoru (počet prvků fronty je dynamicky omezen)
 - pomocí **dynamicky alokovaného a rozšiřitelného pole**
 - pomocí **jednosměrného spojového seznamu**
- Od implementace se očekává, že složitost všech operací bude konstantní (s výjimkou vkládání do rozšiřitelného pole, které se řeší podobně jako v případě zásobníku)

Implementace fronty

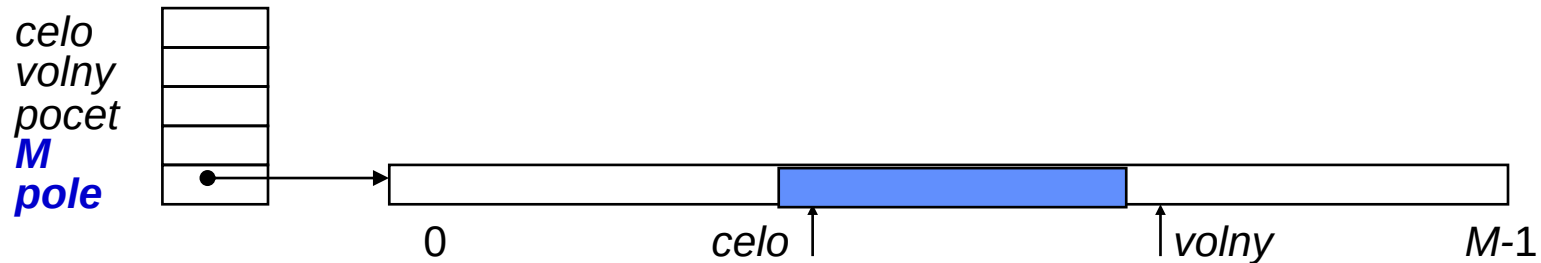
- Pomocí pole se staticky daným počtem prvků



```
class Queue {  
public:  
    enum {M = 40};  
    Queue();  
    void add(const T&);  
    T front() const;  
    void remove();  
    bool empty() const;  
private:  
    int celo, volny, pocet;  
    T pole[M];  
};
```

Implementace fronty

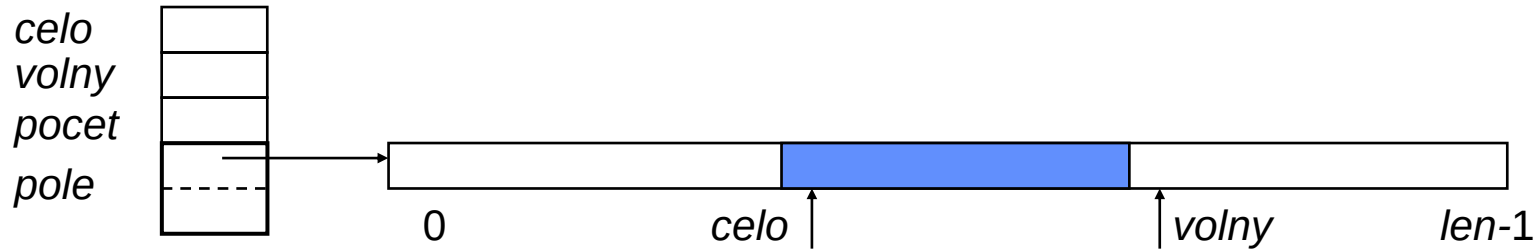
- Pomocí pole s dynamicky daným počtem prvků



```
class Queue {
public:
    Queue(int = 40);
    ~Queue();
    void add(const T&);
    T front() const;
    void remove();
    bool empty() const;
private:
    int celo, volny, pocet;
    int M;
    T* pole;
    Queue(const Queue&);
    Queue& operator=(const Queue&);
};
```

Implementace fronty

- **Pomocí rozšiřitelného pole** (instance šablony třídy *Array*)



```
class Queue {  
public:  
    Queue();  
    bool empty() const {return pocet==0;}  
    void add(const T&);  
    T front() const;  
    void remove();  
private:  
    int celo;  
    int volny;  
    int pocet;  
    Array<T> pole;  
};
```

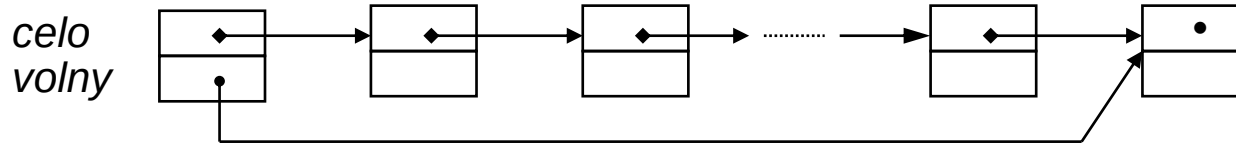
Implementace fronty

- Připomeňme šablonu třídy *Array* pro rozšiřitelné pole

```
template <class Elem>
class Array {
public:
    Array(int=10);
    Array(const Array&);           //konstruktor pole
    ~Array();                     //destruktor pole
    int length() const {return len;} //vrací délku pole
    Elem& operator[] (int);       //přístup k prvku pole
    const Elem& operator[] (int) const;
    Array& operator=(const Array&); //kopírování pole
    void extend(int);             //rozšíření pole
private:
    Elem* array;
    int len;
    void copy(const Array&);
};
```

Implementace fronty

- Pomocí spojového seznamu



```
class Queue {
public:
    Queue();
    ~Queue();
    void add(const T&);
    T front() const;
    void remove();
    bool empty() const;
private:
    struct Item {
        T val;
        Item *next;
        Item() {next = NULL;}
    };
    Item *celo, *volny;
    Queue(const Queue&);
    Queue& operator=(const Queue&);
};
```

ADT Seznam

- Seznam je sekvenční kontejner, pro který operace vložení, odebrání a čtení se provádějí na označené pozici, kterou lze měnit.
- Příkladem seznamu je text zobrazený v okně Poznámkového bloku:
 - pozice je dána umístěním kurzoru
 - stiskem klávesy vložíme znak před kurzor
 - klávesou *Del* vyjmeme znak označný kurzorem
 - klávesou *Backspace* vyjmeme znak před kurzorem
 - klávesou ← resp. → přesuneme kurzor na předchozí resp. následující znak
 - klávesou *Home* resp. *End* přesunem kurzor na začátek resp. konec textu
- Vyzkoušejme ...

ADT Seznam

- Signatura operací nad seznamem (T je typ hodnot v seznamu):

init : $\rightarrow \text{List}$	prázdný seznam
insert (_,_): $\text{List}, T \rightarrow \text{List}$	vlož před kurzor
remove (_): $\text{List} \rightarrow \text{List}$	odeber
removePrev (_): $\text{List} \rightarrow \text{List}$	odeber před kurzorem
toNext (_): $\text{List} \rightarrow \text{List}$	kurzor na další
toPrev (_): $\text{List} \rightarrow \text{List}$	kurzor na předchozí
toBegin (_): $\text{List} \rightarrow \text{List}$	kurzor na začátek
toEnd (_): $\text{List} \rightarrow \text{List}$	kurzor na konec
empty (_): $\text{List} \rightarrow \text{bool}$	je seznam prázdný?
atBegin (_): $\text{List} \rightarrow \text{bool}$	je kurzor na začátku?
atEnd (_): $\text{List} \rightarrow \text{bool}$	je kurzor na konci?
read (_): $\text{List} \rightarrow T$	hodnota na kurzoru

Třída List

```
class List {
public:
    List();
    ~List();
    void insert(T);           // vloz pred kurzor
    void remove();           // odstran prvek na kurzoru
    void removePrev();       // odtstran prvek pred kurzorem
    void toNext();           // kurzor na dalsi prvek
    void toPrev();           // kurzor na predchozi prvek
    void toBegin();          // kurzor na zacatek
    void toEnd();            // kurzor na konec
    bool empty();            // je seznam prazdny?
    bool atBegin();          // je kurzor na zacatku?
    bool atEnd();            // je kurzor na konci?
    bool read(T&);           // cteni z pozice kurzoru
    friend ostream& operator<<(ostream&, const List&);
private:
    ...
};
```

Třída List

- Sémantiku operací si upřesníme na příkladech
- Příklad výpisu seznamu: seznam obsahující hodnoty x, y a z, kde na pozici kurzoru je y, vypíšeme ve tvaru

[x |y z]

- Budeme uvažovat seznam celých čísel

List szn;	[]
szn.insert(10);	[10]
szn.insert(20);	[10 20]
szn.toPrev();	[10 20]
szn.insert(30);	[10 30 20]
szn.toNext();	[10 30 20]
szn.toBegin();	[10 30 20]
szn.toEnd();	[10 30 20]
szn.remove();	[10 30 20]
szn.removePrev();	[10 30]
szn.toPrev();	[10 30]
szn.remove();	[10]
...	

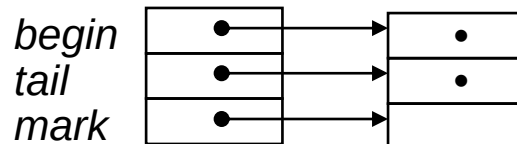
Jakou datovou strukturu použít pro implementaci?

- Požadavek: Chceme, aby všechny operace měly konstantní složitost. Tento požadavek nesplňuje použití pole, kde při vkládání jinak než na konec by bylo třeba posouvat prvky pole (lineární složitost), podobně při odebírání
- Použijeme proto **spojový seznam**
- Objekt typu Spojový seznam bude obsahovat **ukazatel na první prvek** a dále **ukazatel na prvek označený kurzorem**.
- Aby operace „kurzor na předchozí prvek“ měla konstantní složitost, použijeme **dvojsměrný spojový seznam**.
- Aby operace „kurzor na konec“ měla konstantní složitost, bude objekt obsahovat též **ukazatel na konec seznamu**.
- !!! Aby kurzor ukazující na konec seznamu bylo možné posunout na předchozí prvek, bude koncem seznamu poslední prvek spojového seznamu, ve kterém nebude žádná hodnota.

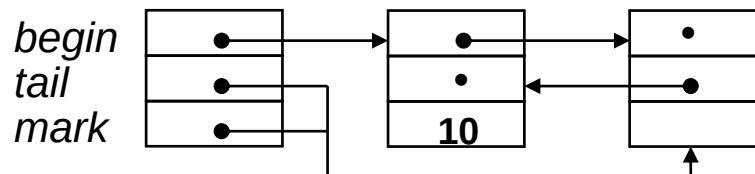
Implementace dvojsměrným spojovým seznamem

- Příklady seznamů a jejich vnitřní reprezentace

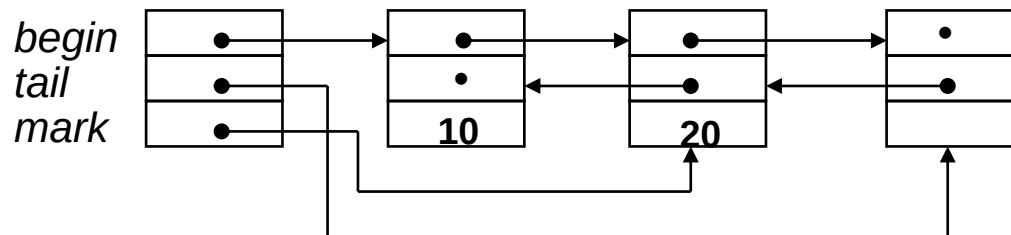
[|]



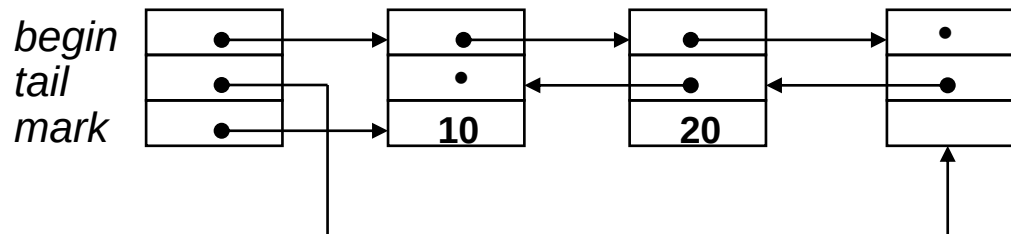
[10 |]



[10 | 20]



[| 10 20]



Privátní část třídy List

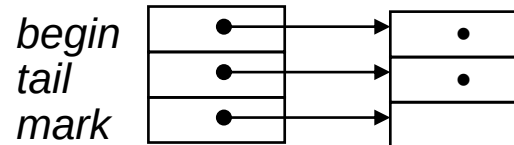
```
private:
    struct Node {
        T val;
        Node *next, *prev;
        Node(T x, Node *n) { // n nesmí být NULL
            val = x; next = n; prev = n->prev;
        }
        Node() {next = NULL; prev = NULL;}
    };
    Node *begin;
    Node *tail;
    Node *mark;
    List(const List &);
    List& operator=(const List&);
};
```

- Konstruktor *Node()* bude použit pro inicializaci uzlu *tail*
- Konstruktor *Node(x, n)* bude použit pro inicializaci uzlu vkládaného před uzel, na který ukazuje *n* (*n* bude vždy ukazatel *mark*, tj, ukazatel na kurzorem označený uzel)

Konstruktor a destruktory třídy List

```
// konstruktor prázdný seznam
List::List() {
    begin = tail = mark = new Node;
}
```

prázdný seznam:



```
// destruktory
List::~~List() {
    Node *p;
    while (begin) {
        p = begin;
        begin = begin->next;
        delete p;
    }
}
```

Testy na prázdnotu a pozici kurzoru

//prázdný seznam?

```
bool List::empty() {  
    return begin == tail;  
}
```

//kurzor na začátku?

```
bool List::atBegin() {  
    return mark==begin;  
}
```

//kurzor na konci?

```
bool List::atEnd() {  
    return mark==tail;  
}
```

Přesuny kurzoru

```
void List::toBegin() {    // kurzor na začátek
    mark = begin;
}

void List::toEnd() {      // kurzor na konec
    mark = tail;
}

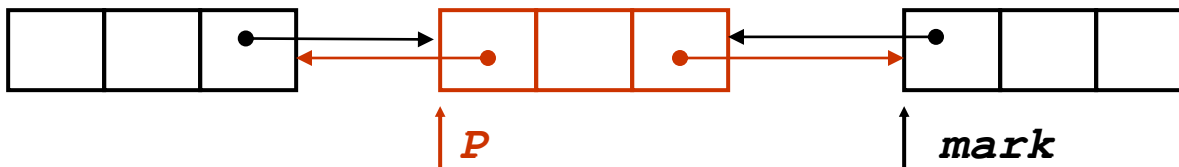
void List::toNext() {     // kurzor na následníka
    if (atEnd()) return;
    mark = mark->next;
}

void List::toPrev() {     // kurzor na předchůdce
    if (atBegin()) return;
    mark = mark->prev;
}
```

Vložení prvku před kurzor

```
void List::insert(T x) {
    Node *p = new Node(x, mark);
    mark->prev = p;
    if (atBegin())
        begin = p;           // vložení na začátek
    else
        p->prev->next = p;
}
```

- Poznámky:
 - položku *prev* v novém prvku nastaví konstruktor (zkopíruje *mark*->*prev*)
 - vkládá-li se před první prvek, je třeba *p* uložit do položky *begin*, jinak do položky *next* přechozího prvku



Odebrání prvku označeného kurzorem

```
void List::remove() {  
    if (atEnd()) return;    //viz poznámka 1  
    Node *p = mark;  
    mark->next->prev = mark->prev;  
    if (atBegin())  
        begin = mark->next;    //ruší se první prvek  
    else  
        mark->prev->next = mark->next;    //viz poznámka 2  
    mark = mark->next;    //viz poznámka 3  
    delete p;  
}
```

- Poznámky:

1. je-li kurzor na konci, neodebere se nic
2. ukazatel na další prvek (*mark->next*) se uloží do *begin*, pokud se odebírá první prvek, jinak do položky *next* předchozího prvku
3. kurzor se posune na prvek za odebraným prvkem

Odebrání prvku před kurzorem

```
void List::removePrev() {  
    if (atBegin()) return;  
    toPrev();  
    remove();  
}
```

- Poznámky:
 - je-li kurzor na začátku, neodebere se nic
 - jinak se kurzor přesune na předchozí prvek a odebere se tento prvek metodou *remove*

Čtení hodnoty prvku a výpis seznamu

```
bool List::read(T &x) {  
    if (atEnd()) return false;  
    x = mark->val;  
    return true;  
}
```

- Poznámka: je-li kurzor na konci, do proměnné x se nic neuloží

```
ostream& operator<<(ostream& s, const List& lst) {  
    s << '[';  
    List::Node *p = lst.begin;  
    while (p) {  
        if (p==lst.mark) s << '|';  
        if (p!=lst.tail) s << p->val << ' ';  
        p = p->next;  
    }  
    s << ']';  
    return s;  
}
```

Třída List

- Testovací program je v souboru p9\list\main.cpp
- Šablona třídy List je v souboru p9/list-sablona/list.h, testovací program pro šablonu je p9\list-sablona\main.cpp
- Vyzkoušejte!

Abstraktní datové typy

- ADT **pole** (Array)
 - specifikace typu
 - mapovací funkce pole
 - šablona pro rozšiřitelné pole, kde přiřazení a kopírující konstruktor používají **techniku počítaných referencí**
- ADT **množina** (Set)
 - základní a množinové operace
 - implementace pomocí charakteristického vektoru
 - implementace neseřazeným polem
 - implementace seřazeným polem
- ADT **tabulka** (Table, Map)
 - základní operace
 - implementace tabulkou s přímým přístupem
 - implementace neseřazeným polem
 - implementace seřazeným polem

ADT Pole

- Pole (array) je **sekvenční kontejner**, který organizuje prvky v n dimenzích a umožňuje rychlý náhodný přístup (random acces) k prvkům pomocí n -tice indexů
- Pole je obecně dáno:
 - typem prvků
 - počtem dimenzí
 - dolní a horní mezí indexu pro každou dimenzi (množiny indexů jsou uspořádané)
- Typ pole lze specifikovat výrazem

array [$d_1 \dots h_1, d_2 \dots h_2, \dots d_N \dots h_N$] **of** T

kde

- T je typ prvků pole
- N je počet dimenzí (indexů)
- d_i je dolní mez i -tého indexu
- h_i je horní mez i -tého indexu

Pro počet prvků pole platí $M = \prod_{1 \leq i \leq N} (h_i - d_i + 1)$

Mapovací funkce pole

- **Mapovací funkce pole**

$$\text{array } [d_1 \dots h_1, d_2 \dots h_2, \dots d_N \dots h_N] \text{ of } T$$

mapuje prvky tohoto pole na prvky pole

array $[0 \dots M - 1]$ of T

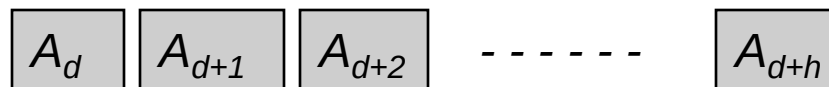
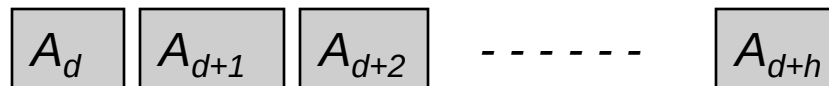
tzn. každé N -tici indexů přiřazuje index z intervalu 0 až $M-1$

Mapovací funkce jednorozměrného pole (N=1)

- array $[d \dots h]$ of T

$$\text{map}(i) = i - d$$

A : array $[d \dots h]$ of T


$$\mathbf{v} : \text{array } [0..d-h-1] \text{ of } T$$


Map(i): 0 1 2 ... M-1=d - h -1

Mapovací funkce dvojrozměrného pole

Uložení matice A po sloupcích

A

A_{11}	A_{12}	A_{13}
A_{21}	A_{22}	A_{23}

V

A_{11}	A_{21}	A_{12}	A_{22}	A_{13}	A_{23}
----------	----------	----------	----------	----------	----------

Uložení matice A po řádcích

A

A_{11}	A_{12}	A_{13}
A_{21}	A_{22}	A_{23}

V

A_{11}	A_{12}	A_{13}	A_{21}	A_{22}	A_{23}
----------	----------	----------	----------	----------	----------

Mapovací funkce pole

- **Mapovací funkce dvojrozměrného pole**

- **array** [$d_1 \dots h_1, d_2 \dots h_2$] of T

$$\text{map}(i_1, i_2) = (\text{počet předcházejících řádků}) \cdot (\text{délka řádku}) + (\text{pozice na řádku}) = (i_1 - d_1)n_2 + (i_2 - d_2)$$

$$\text{kde } n_2 = h_2 - d_2 + 1$$

- Výše uvedené mapování odpovídá uložení prvků v paměti tak, že nejrychleji se mění poslední index (**uložení „po řádcích“**)

- **Mapovací funkce trojrozměrného pole**

- **array** [$d_1 \dots h_1, d_2 \dots h_2, d_3 \dots h_3$] of T

$$\text{map}(i_1, i_2, i_3) = (i_1 - d_1)n_2n_3 + (i_2 - d_2)n_3 + (i_3 - d_3)$$

$$\text{kde } n_2 = h_2 - d_2 + 1, n_3 = h_3 - d_3 + 1$$

- Výše uvedené mapování odpovídá uložení prvků v paměti tak, že nejrychleji se mění poslední index (**uložení „po řádcích“**)

Realizace ADT Pole

- V C++ realizujeme ADT Pole zpravidla pomocí **šablony třídy**
- Základními operacemi jsou:
 - **vytvoření pole** pro daný počet dimenzí N a dané dolní a horní meze indexů
 - **přístup k prvkům pole** pomocí N -tic indexů
- Poznámky k základním operacím:
 - paměť pro uložení prvků pole dynamicky alokuje konstruktor na základě parametrů udávajících dolní a horní meze indexů
 - nesnažme se zadat počet dimenzí pole parametrem konstruktoru, pro každý počet dimenzí vytvořme jinou šablonu třídy (např. *Array1D*, *Array2D*, *Array3D*, atd.)
 - implementace je jednodušší, jestliže ve všech dimenzích je dolní mez indexu 0 (parametry konstruktoru pak mohou udávat pouze horní meze indexů)
 - operace přístupu k prvkům realizujeme pomocí metod nebo přetížených operátorů, ve kterých kontrolujeme přípustnost hodnot indexů
 - implementace je složitější, chceme-li, aby přístup k částím N -rozměrného pole byl dán i menším počtem indexů než je N

Realizace ADT Pole

- Další operace pro ADT Pole
 - vzhledem k dynamické alokaci paměti provedené konstruktorem je třeba definovat **destruktor**
 - je třeba zakázat přiřazení polí nebo definovat **kopírující konstruktor a operátor přiřazení**
 - pole může být **rozšiřitelné**
- Pro kopírující konstruktor a operátor přiřazení lze použít dvě techniky:
 - **hlubokou kopii**
 - **počítané reference**
- Techniku hluboké kopie známe a několikrát jsme ji použili:
 - při přiřazení nebo inicializaci objektu kopírujícím konstruktorem se dynamicky alokovaná data zkopírují do nové, dynamicky alokované paměti
- Technika počítaných referencí
 - k dynamicky alokované paměti se připojí počítadlo objektů, které tuto paměť používají (referencují), a hluboká kopie se provede jen tehdy, je-li to nezbytné
- Techniku počítaných referencí ukážeme na příkladu jednorozměrného rozšiřitelného pole s nulovou dolní mezí indexu.

Pole, rozšířené poznámky

Mělká a hluboká kopie

Ing. Josef Vogel, CSc

+ počítané
reference

Katedra softwarového inženýrství

Katedra teoretické informatiky,
Fakulta informačních technologií, ČVUT v Praze

© Karel Müller, Josef Vogel, 2011

Programování a algoritmizace 2,
BI-PA2, 2011, Přednáška 5

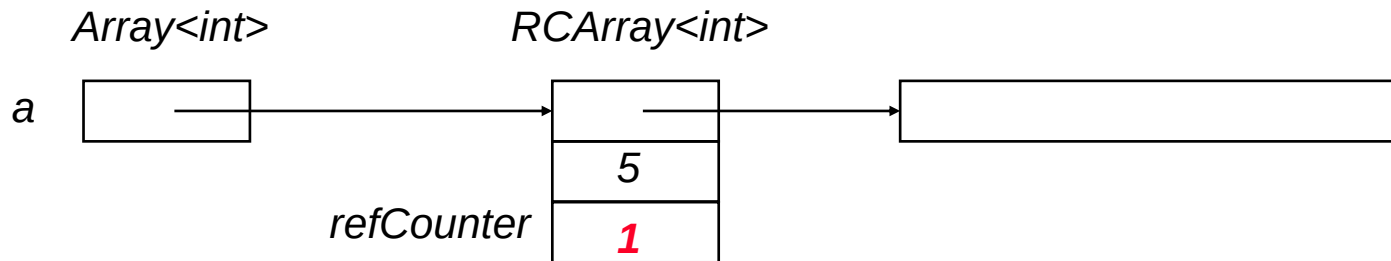


Evropský sociální fond
Praha & EU: Investujeme do vaší budoucnosti

Počítané reference

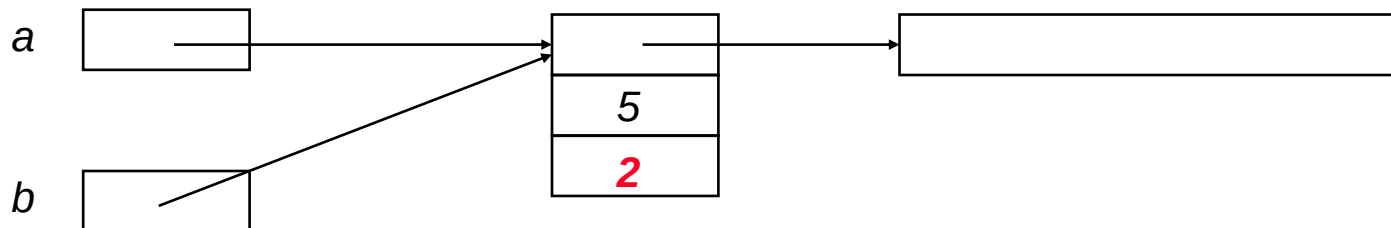
- Jestliže objekt alokující další paměť bude výsledkem nějaké funkce nebo metody, je vhodnější použít místo hluboké kopie techniku *počítaných referencí*.
- Počítané reference pro třídu *Array* – kopírující konstruktor

```
Array<int> a(5);
```



...

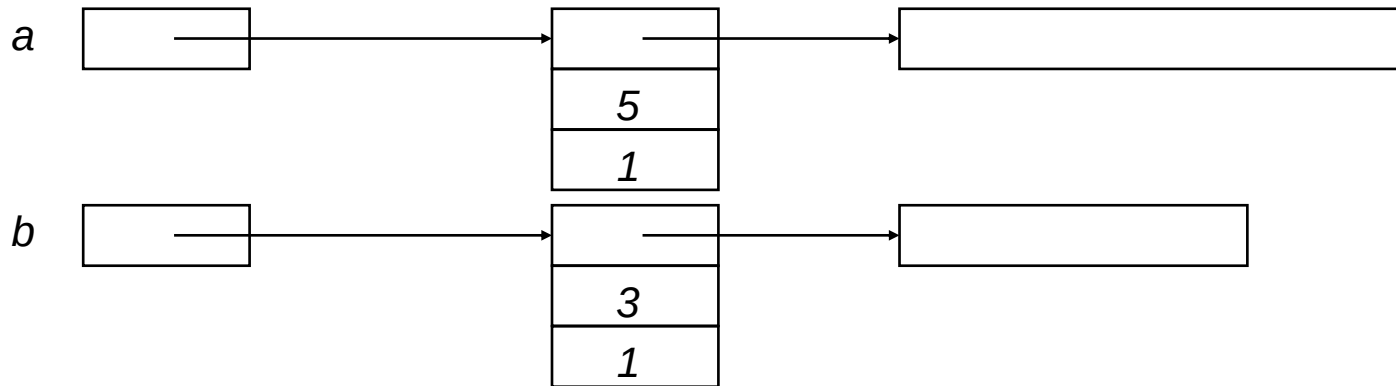
```
Array<int> b=a;
```



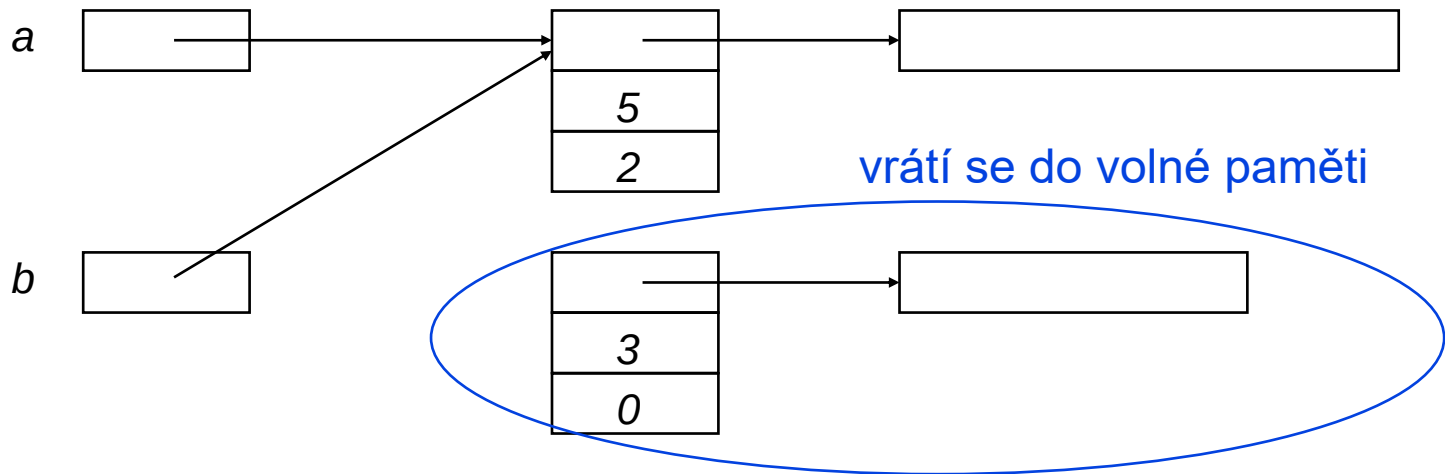
Počítané reference

- Počítané reference pro třídu *Array* – přiřazení

`Array<int> a(5), b(3);`

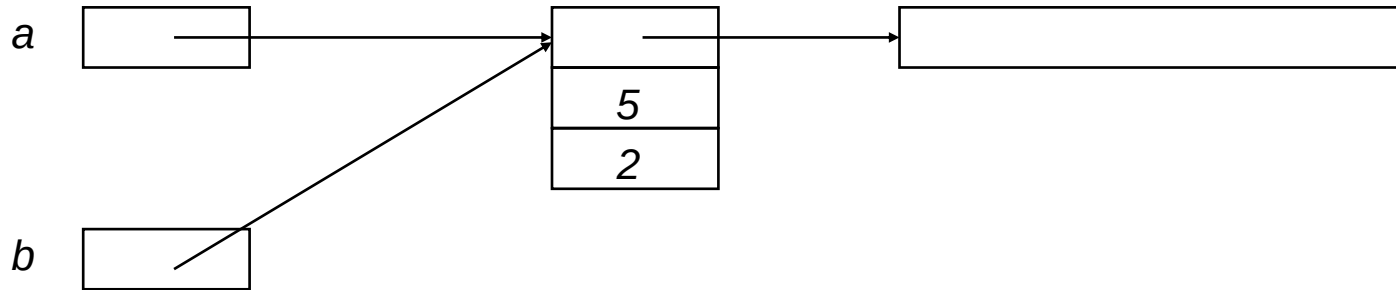


`b = a;`

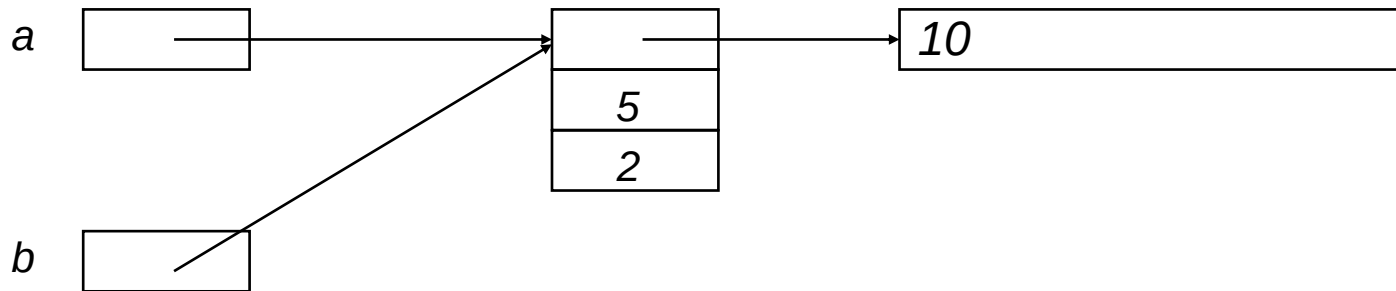


Počítané reference

- Počítané reference pro třídu *Array* – přiřazení prvku pole



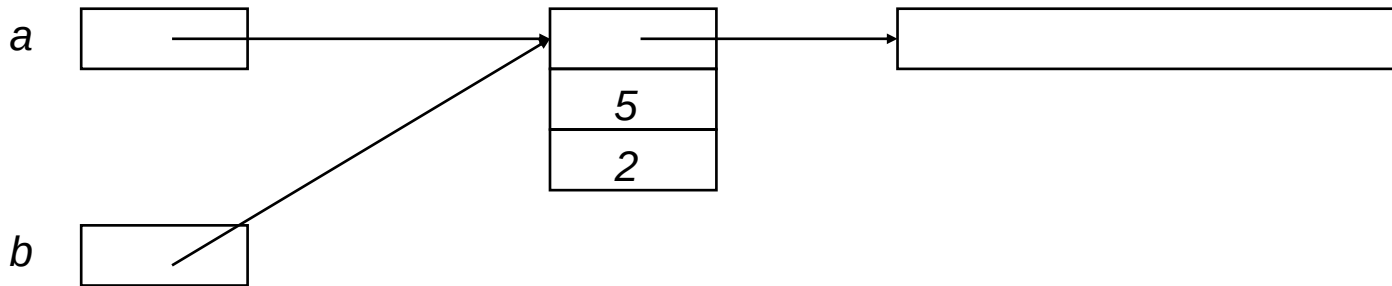
`b[1] = 10;`



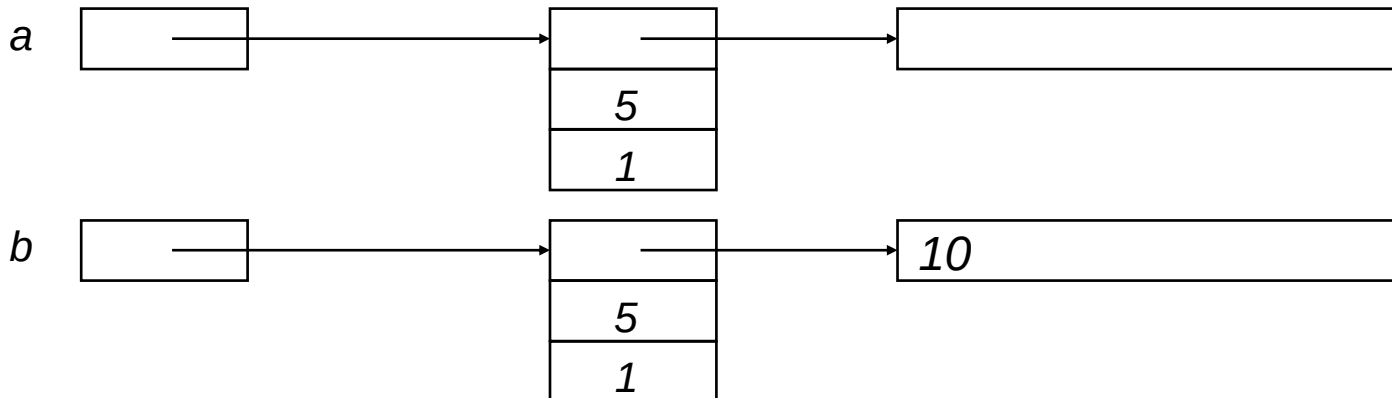
`cout << a[1]; // vypíše 10`

Počítané reference

- !!!! Před přiřazením prvku pole, které je referencováno více objekty, je třeba provést hlubokou kopii
- V operátoru `[]` nelze rozpoznat, zda jde o přiřazení prvku pole, proto je nutné zavést pro **přiřazení prvku pole metodu**



`b.setAt(1,10);`



Šablona třídy Array

- Deklarace třídy:

```
template <class Elem>
class Array {
public:
    Array(int=10);
    Array(const Array&);    /* konstruktor pole */
    ~Array();              /* destruktork pole */
    int length() const;
    const Elem& operator[] (int) const;
    void setAt(int, const Elem&); /*přiřazení prvku pole */
    Array& operator=(const Array&); /*přiřazení pole */
    void extend(int);      /*rozšíření pole */
private:
    struct RArray {
        Elem* array;      //ukazatel na pole prvků
        int len;          //délka pole
        int refCounter;    //počítadlo referencí
        RArray(int l);     //inicializace struktury pole//
    };
};
```

Šablona třídy Array

- Pokračování deklarace třídy:

```
RCArray *rcArray;  
  
// nastaví ukazatel na reprezentující pole  
// a zvýší počítadlo referencí  
void acquire(RCArray* ra) {  
    rcArray = ra;  
    ++ra->refCounter;  
}  
  
// uvolnění pole, když reference klesnou na nulu  
void release() {  
    if (--rcArray->refCounter==0) {  
        delete [] rcArray->array;  
        delete rcArray;  
    }  
}  
};
```

Šablona třídy Array

- Šablony konstruktorů a metod

```
//konstruktor pole
template <class Elem>
Array<Elem>::RCArray::RCArray(int l) {
    len = l;
    array = new Elem[len];
    for (int i=0; i<len; i++) array[i] = Elem();
    refCounter = 1;
}
//konstruktor struktury RCArray
template <class Elem>
Array<Elem>::Array(int lng) {
    rcArray = new RCArray(lng);
}
template <class Elem>
Array<Elem>::Array(const Array<Elem>& a) {
    acquire(a.rcArray);
}
//destruktor pole
template <class Elem>
Array<Elem>::~~Array() {
    release();
}
```

Šablona třídy Array

// čtení délky pole

```
template <class Elem>
    int Array<Elem>::length() const {
        return rcArray->len;           //vrací délku pole
    }
```

//přiřazení prvku pole

```
template <class Elem>
void Array<Elem>::setAt(int ind, const Elem& x) {
    if (ind<0 || ind>=rcArray->len)
        throw "nedovoleny index";
    // vytvoření kopie pole
    if (rcArray->refCounter>1) {
        RCArray *nrcArray = new RCArray(rcArray->len);
        for (int i=0; i<rcArray->len; i++)
            nrcArray->array[i] = rcArray->array[i];
        rcArray->refCounter--;
        rcArray = nrcArray;
    }
    rcArray->array[ind] = x;           //uložení prvku do kopie
}
```

Šablona třídy Array

//čtení prvku pole

```
template <class Elem>
const Elem& Array<Elem>::operator[] (int ind) const {
    if (ind<0 || ind>=rcArray->len)    //kontrola indexu
        throw "nedovoleny index";
    return rcArray->array[ind];
}
```

//přiřazení polí viz sl. 9

```
template <class Elem>    Array<Elem>&
Array<Elem>::operator =(const Array<Elem>& a){
    if (this==&a) return *this;
    //uvolnění pole
    release();
    acquire(a.rcArray);
    return *this;
}
```

Šablona třídy Array

//rozšíření pole

```
template <class Elem>
void Array<Elem>::extend(int nlen) {
//  cout << "extend " << rcArray->len << " -> " << nlen
<< endl;
    if (nlen<=rcArray->len) return;
    //alokace delšího pole
    RArray *nrcArray = new RArray(nlen);
    int i;
    for (i=0; i<rcArray->len; i++ //kopírování hodnot
        nrcArray->array[i] = rcArray->array[i];
    //nulování zbytku
    for (; i<nlen; i++) nrcArray->array[i] = 0;
    release(); //uvolnění původního pole
    rcArray = nrcArray;
//  cout << "extend O.K." << endl;
}
```

//tisk pole

```
template <class Elem>    ostream& operator<<(ostream& s,
const Array<Elem>& a) { ... }
```

ADT Množina

- Vlastnosti množiny známe z předmětu ZTI
- V programování se množinou rozumí kontejner prvků typu T , který neobsahuje duplicity (prvky se stejnou hodnotou) a umožňuje alespoň tyto základní operace:
 - **vložení prvku** do množiny
 - **test příslušnosti** prvku k množině
 - **odebrání prvku**
- Signatura základních operací:

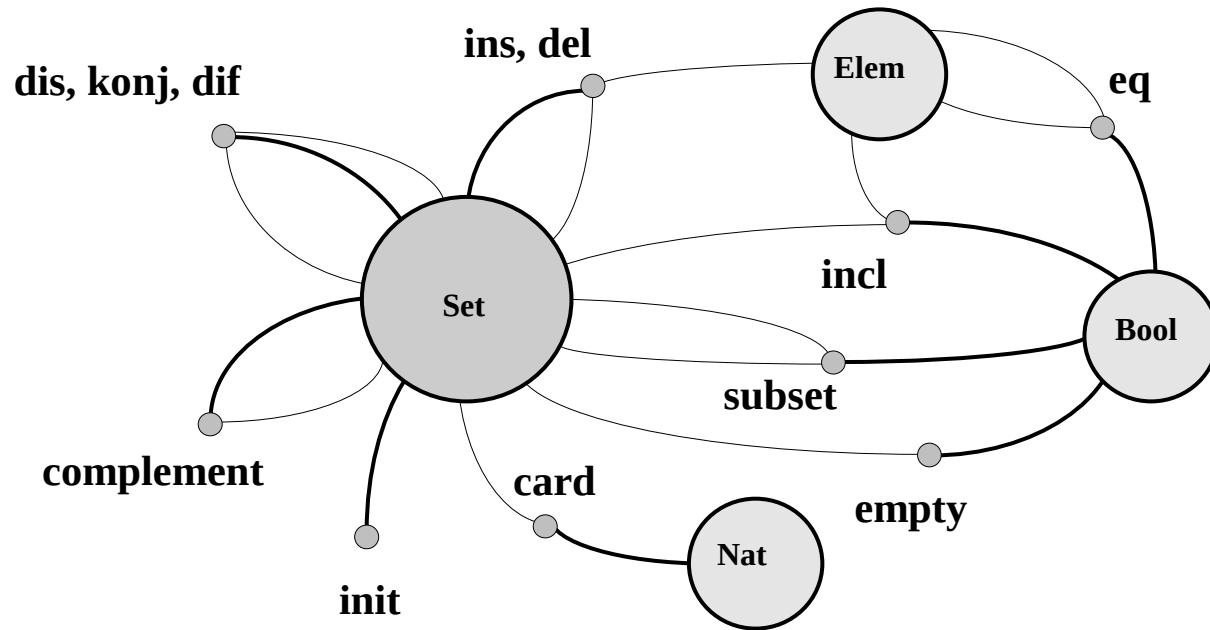
<code>init: -> Set</code>	vytvoření prázdné množiny
<code>ins(_, _): Elem, Set -> Set</code>	vložení prvku do množ.
<code>del(_, _): Elem, Set -> Set</code>	odebrání prvku z množ.
<code>incl(_, _): Elem, Set -> bool</code>	test příslušnosti k množ.
- Někdy jsou potřeba další operace:
 - množinové operace **sjednocení**, **průnik** a **rozdíl**,
 - relační operace **ekvivalence**, **inkluze** atd.
 - výpis množiny

ADT množina (set)

- Statická a dynamická množina
 - Statické množiny: pouze dotaz
 - Dynamické množiny: insert a delete
- Klíčem množiny je hodnota prvku
- Typickou požadovanou funkčností je test příslušnosti prvku k množině
 - Rychlé vyhledávání **konkrétních** hodnot, výpis **unikátních** prvků
 - Používá se i v grafových algoritmech (seznam vrcholů)
- Nemáme přímý přístup k prvkům množiny
- V případě uspořádaných množin nemůžeme měnit hodnoty prvků
 - Modifikace = odstranění staré hodnoty a vložení nové

ADT Množina

Signatura datového typu množina



Implementace ADT Množina

- Probereme několik způsobů implementace množiny
- Každou implementaci realizujeme pomocí třídy nebo šablony třídy
- V následující deklaraci jsou uvedeny operace, kterými se budeme zabývat:

```
class Set {  
public:  
    Set(); // inicializace prázdné množiny  
    void ins(T); // vložení prvku  
    void del(T); // odebrání prvku  
    bool incl(T) const; // test příslušnosti  
    Set operator+(const Set&) const; // sjednocení  
    Set operator*(const Set&) const; // průnik  
    bool operator==(const Set&) const; // test rovnosti  
    bool operator<=(const Set&) const; // test inkluze  
    friend ostream& operator<<(ostream&, const Set&);  
private:  
    ...  
};
```

- Pro každý způsob implementace odhadneme složitost operací

Implementace ADT Množina

- Pro implementaci množiny lze použít:
 - **charakteristický vektor** množiny (bitové pole)
 - **neseřazené pole**
 - **seřazené pole**
 - jednosměrný **spojový seznam**
 - **uspořádaný spojový seznam**
 - binární **vyhledávací strom**
 - **rozptylovací (hash) tabulku**

Implementace množiny charakteristickým vektorem

- Nejprve něco z matematiky:

Mějme nosnou množinu $X = \{x_1, x_2, \dots, x_n\}$. Pro $A \subseteq X$ definujeme **charakteristický vektor** χ_A jako

$$\chi_A = \{c_1, c_2, \dots, c_n\}, \text{ kde } c_i = 1 \text{ pro } x_i \in A \text{ a } c_i = 0 \text{ jinak, } 1 \leq i \leq n.$$

- Tvoří-li nosnou množinu interval celých čísel od *min* do *max*, pak množinu můžeme implementovat pomocí pole s prvky typu *bool* a indexy od *min* do *max*
- Pro zjednodušení implementace množinových operací budeme předpokládat *min* = 0, tzn. použijeme pole s indexy od 0 do *max*

Implementace množiny charakteristickým vektorem

- Implementaci operací nejprve popíšeme slovně s použitím této notace:

$a.pole$ pole reprezentující charakteristický vektor množiny a

$a.pole[x]$ prvek pole pro hodnotu x

n_a velikost (počet prvků) $a.pole$
(počet prvků množiny je obvykle menší)

- Základní operace:

operace	implementace	složitost
$a.ins(x)$	$a.pole[x] = true$	$O(1)$
$a.del(x)$	$a.pole[x] = false$	$O(1)$
$a.incl(x)$	výsledkem je $a.pole[x]$	$O(1)$

Implementace množiny charakteristickým vektorem

- Množinové operace:

operace	implementace	složitost
$a + b$	vytvoříme novou množinu kopií množiny a , do které vložíme každý prvek množiny b	$O(n_a + n_b)$
$a * b$	vytvoříme novou prázdnou množinu, do které vložíme každý prvek množiny a , který je obsažen v množině b	$O(n)$
$a == b$	výsledkem je true, když každý prvek $a.pole[i]$ má stejnou hodnotu jako $b.pole[i]$	$O(n)$
$a \leq b$	výsledkem je true, když každý prvek množiny a je také prvkem množiny b	$O(n)$

- Poznámky:
 - $n = \max(n_a, n_b)$
 - složitost provedení operace pro všechny prvky množiny a je $O(n_a)$

Třída Set

- Množinu implementovanou charakteristickým vektorem můžeme realizovat třídou Set, ve které použijeme šablonu pro rozšiřitelné pole:

```
// set.h
#include "array.h"

class Set {
public:
    explicit Set(int=0);
    void ins(int);
    void del(int);
    bool incl(int) const;
    Set operator+(const Set&) const;
    Set operator*(const Set&) const;
    bool operator==(const Set&) const;
    bool operator<=(const Set&) const;
    friend ostream& operator<<(ostream&, const Set&);
    friend Set set(int=-1, ...);
private:
    Array<bool> arr;
};
```

Metody třídy Set

```
Set::Set(int m) : arr(m+1) {}

void Set::ins(int x) {
    if (x >= arr.length())
        arr.extend(max(x+1, 2*arr.length())); //rozšíření
    try {arr.setAt(x, 1);}                     //vlození
    catch (...) {}
}

void Set::del(int x) {                          //rušení prvku
    try {arr.setAt(x, 0);}
    catch (...) {}
}

bool Set::incl(int x) const {                  //existuje prvek?
    try {return arr[x]; }
    catch (...) {return false;}
}
```

Metody třídy Set

```
Set Set::operator+ (const Set& y) const {
    int nx = arr.length(), ny = y.arr.length();
    if (nx<ny) return y + *this;
    Set res = *this;           // delší množina do výsledku
    for (int i=0; i<ny; i++)
        // přidej prvky kratší množiny
        if (y.incl(i)) res.ins(i);
    return res;
}
```

```
Set Set::operator* (const Set& y) const {
    int nx = arr.length(), ny = y.arr.length();
    if (nx>ny) return y * *this;           //výběr kratší
    Set res;                               // inicilizace výsledné množiny
    for (int i=0; i<nx; i++)
        if (incl(i) && y.incl(i)) res.ins(i);
    return res;
}
```

Metody třídy Set

// ekvivalence množin

```
bool Set::operator==(const Set& y) const {  
    int nx = arr.length(), ny = y.arr.length();  
    if (nx>ny) return y==*this;           //výběr kratší  
    for (int i=0; i<nx; i++)  
        if (arr[i]!=y.arr[i]) return false;    //xi ≠ yi  
    for (int i=nx; i<ny; i++)  
        if (y.arr[i]) return false;    //yi není prázdný  
    return true;  
}
```

// inkluze množin

```
bool Set::operator<= (const Set& y) const {  
    int n = arr.length();  
    for (int i=0; i<n; i++)    //je prvek xi prvkem yi?  
        if (arr[i] && !y.incl(i)) return false;  
    return true;  
}
```

Metody třídy Set

```
// tisk množiny
```

```
ostream& operator<<(ostream& s, const Set& m) {  
    bool mezera = false;  
    s << '[';  
    for (int i=0, n=m.arr.length(); i<n; i++)  
        if (m.arr[i]) {  
            s << (mezera ? " " : "") << i;  
            mezera = true;  
        }  
    s<<']';  
    return s;  
}
```

Metody třídy Set

//inicializace množiny hodnotami parametrů

```
Set set(int x, ...) {  
    va_list ppar;  
    Set res;  
    if (x<0) return res;  
    res.ins(x);  
    va_start(ppar, x);  
    for (;;) {  
        x = va_arg(ppar, int);  
        if (x<0) break;  
        res.ins(x);  
    }  
    va_end(ppar);  
    return res;  
}
```

Použití třídy Set

- Soubor \p10\set\main1.cpp
- Soubor \p10\set\sito.cpp ilustruje použití třídy Set pro Eratosthenovo síto

```
const int N = 1000;
int main() {
    int i, j;
    Set a(N+1);
    for (int i=2; i<=N; i++) a.ins(i);
    int p = 2, pmax = (int)sqrt(N);
    do {
        // vypusteni vsech nasobku cisla p
        for (int i= p+p; i<=N; i+=p) a.del(i);
        // hledani nejbližsiho cisla k p
        do {
            p++;
        } while (!a.incl(p));
    } while (p<=pmax);
    cout << a << endl;
    system("PAUSE");
    return 0;
}
```

Implementace množiny neseřazeným polem

- Princip: Prvky množiny se ukládají do neseřazeného pole (dynamicky alokovaného)
- Slovní popis základních operací:

operace	implementace	složitost
$a.ins(x)$	vložení x na první volné místo	$O(1)$
$a.del(x)$	sekvenční hledání prvku s hodnotou x , pak posun dalších prvků o jedno místo doleva	$O(n_a)$
$a.incl(x)$	sekvenční hledání prvku pole s hodnotou x	$O(n_a)$

- Poznámka:
 n_a je počet prvků množiny a (obvykle je to méně, než počet prvků pole)

Implementace množiny neseřazeným polem

- Slovní popis množinových operací:

operace	implementace	složitost
$a + b$	vytvoříme novou množinu kopií množiny a , do které vložíme každý prvek množiny b	$O(n_a n_b)$
$a * b$	vytvoříme novou prázdnou množinu, do které vložíme každý prvek množiny a , který je též v množině b	$O(n_a n_b)$
$a == b$	výsledkem je true, když množiny mají stejný počet prvků a každý prvek množiny a je též prvkem množiny b	$O(n_a n_b)$
$a \leq b$	výsledkem je true, když každý prvek množiny a je též prvkem množiny b	$O(n_a n_b)$

Šablona třídy ArrSet

```
template <class T>
class ArrSet {
public:
    explicit ArrSet(int=10);
    void ins(const T&);
    void del(const T&);
    bool incl(const T&) const;
    ArrSet operator+(const ArrSet&) const;
    ArrSet operator*(const ArrSet&) const;
    bool operator==(const ArrSet&) const;
    bool operator<=(const ArrSet&) const;
    ostream& print(ostream&) const;
private:
    Array<T> arr;           // rozsiritelne pole
    int num;               // pocet prvku v mnozine
    int search(const T&) const; //vrati index prvku nebo -1
};
```

Šablony metod třídy ArrSet

```
// konstruktor množiny
template <class T>
ArrSet<T>::ArrSet(int m) : arr(m), num(0) {}

// sekvenční hledání v poli
template <class T>
int ArrSet<T>::search(const T& x) const {
    for (int i=0; i<num; i++)
        if (arr[i]==x) return i;
    return -1;
}

//vkládání do rozšiřitelného pole
template <class T>
void ArrSet<T>::ins(const T& x) {
    if (search(x)>=0) return;
    if (num>=arr.length())
        arr.extend(2*arr.length());
    arr.setAt(num++, x);
}
```

Šablony metod třídy ArrSet

```
// rušení prvku
template <class T>
void ArrSet<T>::del(const T& x) {
    int k = search(x);           // hledej prvek x
    if (k<0) return;             // prvek není v množině
    for (k++; k<num; k++)        // posun prvků
        arr.setAt(k-1, arr[k]);
    num--;
}

//je prvkem množiny?
template <class T>
bool ArrSet<T>::incl(const T& x) const {
    return search(x)>=0;         // sekvenční hledání x
}
```

Šablony metod třídy ArrSet

//sjednocení množin

```
template <class T>
```

```
ArrSet<T> ArrSet<T>::operator+ (const ArrSet<T>& y) const  
{  
    ArrSet<T> res = *this;    //kopie prvního operandu do res  
    for (int i=0; i<y.num; i++)  
        y.incl(y.arr[i]);  
    return res;  
}
```

//průnik množin

```
template <class T>
```

```
ArrSet<T> ArrSet<T>::operator* (const ArrSet<T>& y) const  
{  
    ArrSet res;  
    for (int i=0; i<num; i++)  
        if (y.incl(arr[i])) res.ins(arr[i]);  
    return res;  
}
```

Šablony metod třídy ArrSet

//ekvivalence množin

```
template <class T>
```

```
bool ArrSet<T>::operator==(const ArrSet<T>& y) const {  
    if (num!=y.num) return false; // různý počet prvků  
    for (int i=0; i<num; i++)  
        if (!y.incl(arr[i])) return false;  
    return true;  
}
```

//inkluze množin

```
template <class T>
```

```
bool ArrSet<T>::operator<=(const ArrSet<T>& y) const {  
    for (int i=0; i<num; i++)  
        if (!y.incl(arr[i])) return false;  
    return true;  
}
```

Implementace množiny seřazeným polem

- Princip: Prvky množiny se ukládají do seřazeného pole (dynamicky alokovaného)
- Slovní popis základních operací:

operace	implementace	složitost
$a.ins(x)$	vložení x do seřazeného pole	$O(n_a)$
$a.del(x)$	binární hledání prvku s hodnotou x , pak posun dalších prvků o jedno místo doleva	$O(n_a)$
$a.incl(x)$	binární hledání prvku pole s hodnotou x	$O(\log(n_a))$

- Poznámka:
 n_a je počet prvků množiny a (obvykle je to méně, než počet prvků pole)

Implementace množiny seřazeným polem

- Slovní popis množinových operací:

operace	implementace	složitost
$a + b$	vytvoříme novou množinu kopií množiny a , do které vložíme každý prvek množiny b	$O(n_a \cdot n_b)$
$a * b$	vytvoříme novou prázdnou množinu, do které vložíme každý prvek množiny a , který je též v množině b	$O(n_a \cdot n_b)$
$a == b$	výsledkem je true, když množiny mají stejný počet prvků a každý prvek množiny a je též prvkem množiny b	$O(n_a \log(n_b))$
$a \leq b$	výsledkem je true, když každý prvek množiny a je též prvkem množiny b	$O(n_a \log(n_b))$

Šablona třídy SrtArrSet

```
template <class T>
class SrtArrSet {
public:
    explicit SrtArrSet(int=10);
    void ins(const T&);
    void del(const T&);
    bool incl(const T&) const;
    SrtArrSet operator+(const SrtArrSet&) const;
    SrtArrSet operator*(const SrtArrSet&) const;
    bool operator==(const SrtArrSet&) const;
    bool operator<=(const SrtArrSet&) const;
    ostream& print(ostream&) const;
private:
    Array<T> arr;    // rozsiritelne pole
    int num;         // pocet prvku v mnozine
    int search(const T&) const; //vrati index prvku nebo -1
}
```

Šablony metody třídy SrtArrSet

```
template <class T>
SrtArrSet<T>::SrtArrSet(int m) : arr(m), num(0) {}
```

```
template <class T>
int SrtArrSet<T>::search(const T& x) const {
    int d = 0, h=num-1, k;
    // hledání binárním půlením
    while (d<=h) {
        k = (d+h)/2;
        if (x<arr[k]) h = k-1;
        else if (x>arr[k]) d = k+1;
        else return k;
    }
    return -1;
}
```

Šablony metody třídy SrtArrSet

```
template <class T>                                // vložení prvku
void SrtArrSet<T>::ins(const T& x) {
    if (search(x)>=0) return;
    if (num>=arr.length())
        arr.extend(2*arr.length());              // rozšíření pole

    // zatřídění nového prvku postupem od konce pole arr
    int k=num-1;
    while (k>=0 && arr[k]>x) {
        arr.setAt(k+1, arr[k]);                   // posun prvků
        k--;
    }
    arr.setAt(k+1, x);                             // vložení prvku
    num++;
}
```

Šablony metody třídy SrtArrSet

```
template <class T>                                //rušení prvku
void SrtArrSet<T>::del(const T& x) {
    int k = search(x);                            // nalezení prvku
    if (k<0) return;
    for (k++; k<num; k++)
        arr.setAt(k-1, arr[k]);                  //posun větších prvků
    num--;
}

template <class T>                                //je prvkem množiny?
bool SrtArrSet<T>::incl(const T& x) const {
    return search(x)>=0;
}
```

Šablony metody třídy SrtArrSet

```
template <class T>                                //sjednocení množin
SrtArrSet<T> SrtArrSet<T>::operator+
            (const SrtArrSet<T>& y) const {
    SrtArrSet<T> res = *this;
    for (int i=0; i<y.num; i++)
        y.incl(y.arr[i]);
    return res;
}
```

```
template <class T>                                //průnik množin
SrtArrSet<T> SrtArrSet<T>::operator*
            (const SrtArrSet<T>&y) const {
    SrtArrSet res;
    for (int i=0; i<num; i++)
        if (y.incl(arr[i])) res.ins(arr[i]);
    return res;
}
```

Šablony metody třídy SrtArrSet

```
template <class T>                                //ekvivalence množin
bool SrtArrSet<T>::operator==(const SrtArrSet<T>& y)
const {
    if (num!=y.num) return false;
    for (int i=0; i<num; i++)
        if (!y.incl(arr[i])) return false;
    return true;
}
```

```
template <class T>                                //inkluze množin
bool SrtArrSet<T>::operator<= (const SrtArrSet<T>& y)
const {
    for (int i=0; i<num; i++)
        if (!y.incl(arr[i])) return false;
    return true;
}
```

ADT Tabulka

- Tabulku (table, map) lze charakterizovat jako zobrazení z množiny *klíčů* do množiny *hodnot*
- ADT Tabulka je asociativní kontejner obsahující dvojice *klíč-hodnota*
- Signatura základních operací:

init: -> Tab

ins(⟦,⟦, ⟦): Key, Value, Tab -> Tab

del(⟦,⟦): Key, Tab -> Tab

incl(⟦,⟦): Key, Tab -> bool

read((⟦,⟦): Key, Tab -> Value

vytvoření prázdné tabulky

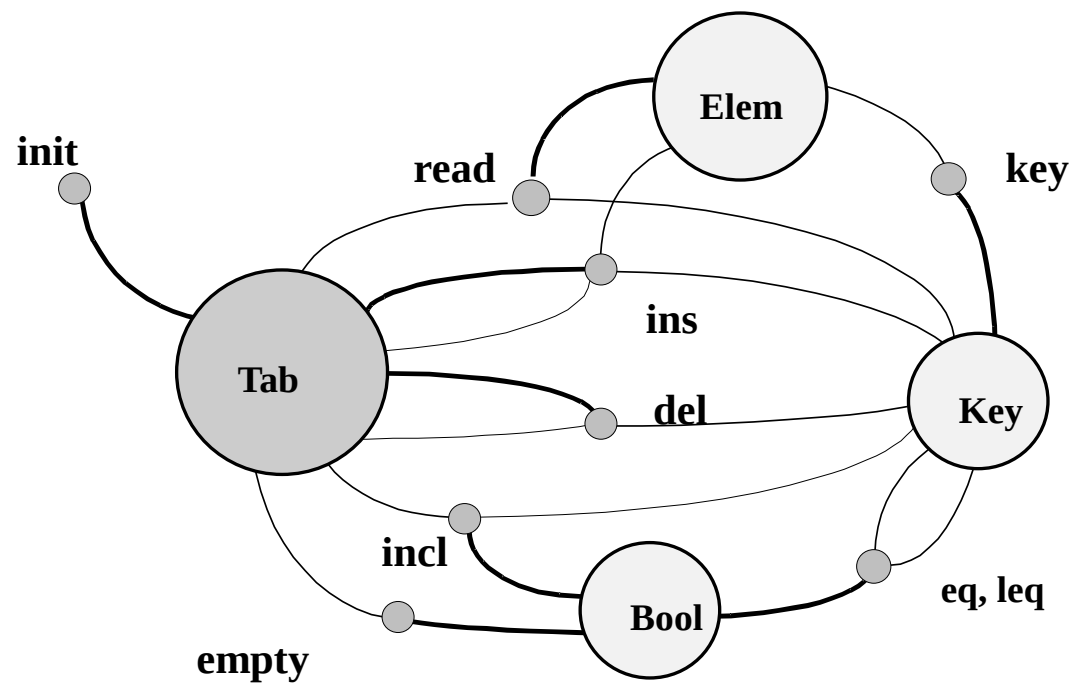
vložení prvku s daným klíčem
a hodnotou.

odebrání prvku s daným klíčem

test, zda tabulka obsahuje prvek
s daným klíčem

čtení hodnoty prvku s daným klíčem

Signatura datového typu Tabulka



Implementace ADT Tabulka

- Pro implementaci tabulky lze použít podobné techniky, jako pro množinu
Poznámka: množina je vlastně tabulka, která klíčům přiřazuje hodnoty *true* nebo *false*
- Pro realizaci tabulky použijeme šablonu třídy
- Následující deklarace ukazuje, jaké metody zavedeme pro operace s tabulkou

```
class Tab {  
    Tab(); // inicializace prázdné tabulky  
    void ins(Key, Val); // vložení prvku do tabulky  
    void del(Key); // odebrání prvku z tabulky  
    bool incl(Key); // test existence prvku  
    bool read(Key, Val&); // čtení hodnoty prvku  
};
```

Implementace ADT Tabulka

- Pro implementaci tabulky lze použít:
 - **pole**, jehož indexy jsou klíče (**tabulka s přímým přístupem**)
 - **neseřazené pole**
 - **seřazené pole**
 - **jednosměrný spojový seznam**
 - **uspořádaný spojový seznam**
 - binární **vyhledávací strom**
 - **rozptylovací (hash) tabulku**

Tabulka s přímým přístupem

- Jsou-li klíči celá čísla od 0 do max , použijeme pro uložení hodnot pole, jehož indexy jsou klíče.
Prvek tabulky s klíčem k tedy bude mít hodnotu uloženou v prvku pole $a[k]$
- Pro neceločíselné klíče najdeme jednoznačné zobrazení *klíč* $\rightarrow$ *index*
- Je třeba definovat prázdnou položku:
z množiny možných hodnot vyčleníme jednu, která bude znamenat, že „klíč nemá přiřazenou žádnou hodnotu (např. pro množinu hodnot typu *string* to může být prázdný řetězec. Tuto hodnotu označíme ***none***“
- Slovní popis operací:

operace	implementace	složitost
$a.ins(k, v)$	$a.pole[k] = v$	$O(1)$
$a.del(k)$	$a.pole[k] = none$	$O(1)$
$a.incl(k)$	výsledkem je true, jestliže $a.pole[k] \neq none$	$O(1)$
$a.read(k, x)$	$x = a.pole[k]$	$O(1)$

Šablona třídy DirAccTab

- Parametrem šablony třídy *DirAccTab* bude typ hodnot *V* a konstanta ***none*** typu *V*, která bude hodnotou prvku pole v případě, že klíč nemá přiřazenou hodnotu
- Pro uložení hodnot opět použijeme šablonu třídy realizující rozšiřitelné pole

```
template <class V, V none>
class DirAccTab {
public:
    explicit DirAccTab(int=10);
    void ins(int, const V&);
    void del(int);
    bool incl(int) const;
    bool read(int, V&) const;
    ostream& print(ostream&) const;
private:
    Array<V> arr;    // rozsiritelne pole
};
```

Šablony metod třídy DirAccTab

```
template <class V, V none>          //inicializace tabulky
DirAccTab<V,none>::DirAccTab(int m) : arr(m+1) {
    for (int i=0; i<arr.length(); i++)
        arr.setAt(i,none);
}

template <class V, V none>          //vložení prvku do tabulky
void DirAccTab<V,none>::ins(int k, const V& v) {
    if (k>=arr.length())
        arr.extend(max(k+1, 2*arr.length()));
    try {arr.setAt(k,v);}
    catch (...) {}
}

template <class V, V none>          //rušení prvku
void DirAccTab<V,none>::del(int k) {
    try {arr.setAt(k,none);}
    catch (...) {}
}
```

Šablony metod třídy DirAccTab

```
template <class V, V none>           //test existence prvku
bool DirAccTab<V,none>::incl(int k) const {
    try {return arr[k]!=none;}
    catch (...) {return false;}
}

template <class V, V none>           //čtení prvku
bool DirAccTab<V,none>::read(int k, V& v) const {
    try {
        if (arr[k]!=none) {
            v = arr[k]; return true;
        } else return false;
    }
    catch (...) {return false;}
}
```

- Použití je v souboru p10\table\main1.cpp

Implementace tabulky neseřazeným polem

- Do neseřazeného pole se ukládají dvojice *klíč-hodnota*
- Slovní popis základních operací:

operace	implementace	složitost
$a.ins(k,v)$	vložení dvojice $\langle k,v \rangle$ na první volné místo	$O(1)$
$a.del(k)$	sekvenční hledání prvku s klíčem k , pak posun dalších prvků o jedno místo doleva	$O(n)$
$a.incl(k)$	sekvenční hledání prvku pole s klíčem k	$O(n)$
$a.read(k,x)$	sekvenční hledání prvku pole s klíčem k a pak uložení hodnoty do proměnné x	$O(n)$

- Realizovat budeme šablonou třídy *ArrTab*, která bude mít parametr K označující typ klíčů a V označující typ hodnot
- Pro dvojice *klíč-hodnota* zavedeme šablonu třídy **TabElem** s parametry K a V

Šablona třídy ArrTab

```
template <class K, class V>
    struct TabElem { //struktura prvku tabulky
        K key;
        V val;
        TabElem(K k, V v) : key(k), val(v) {}
        TabElem() {}
    };
template <class K, class V>
class ArrTab {
public:
    explicit ArrTab(int=10);
    void ins(const K&, const V&);
    void del(const K&);
    bool incl(const K&) const;
    bool read(const K&, V&) const;
    ostream& print(ostream&) const;
private:
    Array<TabElem<K,V> > arr; //rozsiritelne pole
    int num; // pocet prvku v tabulce
    int search(const K&) const; //vrati index prvku nebo -1
};
```

Šablony metod třídy ArrTab

```
template <class K, class V> // konstruktor
ArrTab<K,V>::ArrTab(int m) : arr(m), num(0) {}

template <class K, class V> //sekvenční hledání prvku
int ArrTab<K,V>::search(const K& k) const {
    for (int i=0; i<num; i++)
        if (arr[i].key==k) return i;
    return -1;
}

template <class K, class V> // vložení prvku
void ArrTab<K,V>::ins(const K& k, const V& v) {
    int j = search(k);
    if (j>=0) { //přepsání existujícího prvku
        arr.setAt(j, TabElem<K,V>(k,v));
        return;
    }
    if (num>=arr.length()) //tabukla je plná
        arr.extend(2*arr.length());
    //vložení nového prvku
    arr.setAt(num++, TabElem<K,V>(k,v));
}
```

Šablony metod třídy ArrTab

```
template <class K, class V>                                //rušení prvku
void ArrTab<K,V>::del(const K& k) {
    int j = search(k);
    if (j<0) return;
    for (j++; j<num; j++)                                //přesun prvků
        arr.setAt(j-1, arr[j]);
    num--;
}

template <class K, class V>                                //existuje prvek?
bool ArrTab<K,V>::incl(const K& k) const {
    return search(k)>=0;
}

template <class K, class V>                                //čtení prvku
bool ArrTab<K,V>::read(const K& k, V& v) const {
    int j = search(k);
    if (j<0) return false;
    v = arr[j].val;                                       //výběr hodnoty prvku
    return true;
}
```

Implementace tabulky seřazeným polem

- Do seřazeného pole se ukládají dvojice *klíč-hodnota*
- Slovní popis základních operací:

operace	implementace	složitost
$a.ins(k,v)$	vložení dvojice $\langle k,v \rangle$ do seřazeného pole	$O(n)$
$a.del(k)$	binární hledání prvku s klíčem k , pak posun dalších prvků o jedno místo doleva	$O(n)$
$a.incl(k)$	binární hledání prvku pole s klíčem k	$O(\log(n))$
$a.read(k,x)$	binární hledání prvku pole s klíčem k a pak uložení hodnoty do proměnné x	$O(\log(n))$

- Realizovat budeme šablonou třídy *SrtArrTab*, která bude mít parametr K označující typ klíčů a V označující typ hodnot
- Pro dvojice *klíč-hodnota* zavedeme šablonu třídy *TabElem* s parametry K a V

Šablona třídy SrtArrTab

```
template <class K, class V>
    struct TabElem {
        K key;
        V val;
        TabElem(K k, V v) : key(k), val(v) {}
        TabElem() {}
    };

template <class K, class V>
class SrtArrTab {
public:
    explicit SrtArrTab(int=10);
    void ins(const K&, const V&);
    void del(const K&);
    bool incl(const K&) const;
    bool read(const K&, V&) const;
    ostream& print(ostream&) const;
private:
    Array<TabElem<K,V> > arr;    // rozsiritelne pole
    int num;                    // pocet prvku v tabulce
    int search(const K&) const; //vrati index prvku nebo -1
};
```

Šablony metod třídy SrtArrTab

```
template <class K, class V>      // konstruktor
SrtArrTab<K,V>::SrtArrTab(int m) : arr(m), num(0) {}

template <class K, class V>
int SrtArrTab<K,V>::search(const K& k) const {
    int d = 0, h=num-1, j;
    while (d<=h) {                //hledání binárním půlením
        j = (d+h)/2;
        if (k<arr[j].key) h = j-1;    //dolní polovina
        else if (k>arr[j].key) d = j+1; //horní polovina
        else return j;
    }
    return -1;
}
```

Šablony metod třídy SrtArrTab

```
template <class K, class V>                // vložení prvku
void SrtArrTab<K,V>::ins(const K& k, const V& v) {
    int j = search(k);
    if (j>=0) {                            //přepsání existujícího prvku
        arr.setAt(j, TabElem<K,V>(k,v));
        return;
    }
    if (num>=arr.length())
        arr.extend(2*arr.length());      // rozšíření pole

    //posun prvků počínaje num-1
    j=num-1;
    while (j>=0 && arr[j].key>k) {
        arr.setAt(j+1, arr[j]);
        j--;
    }
    //vložení nového prvku
    arr.setAt(j+1, TabElem<K,V>(k,v));
    num++;
}
```

Šablony metod třídy SrtArrTab

```
template <class K, class V>      //rušení prvku
void SrtArrTab<K,V>::del(const K& k) {
    int j = search(k);
    if (j<0) return;              //nenalezen
    for (j++; j<num; j++)         //posun prvků počínaje j+1
        arr.setAt(j-1, arr[j]);
    num--;
}

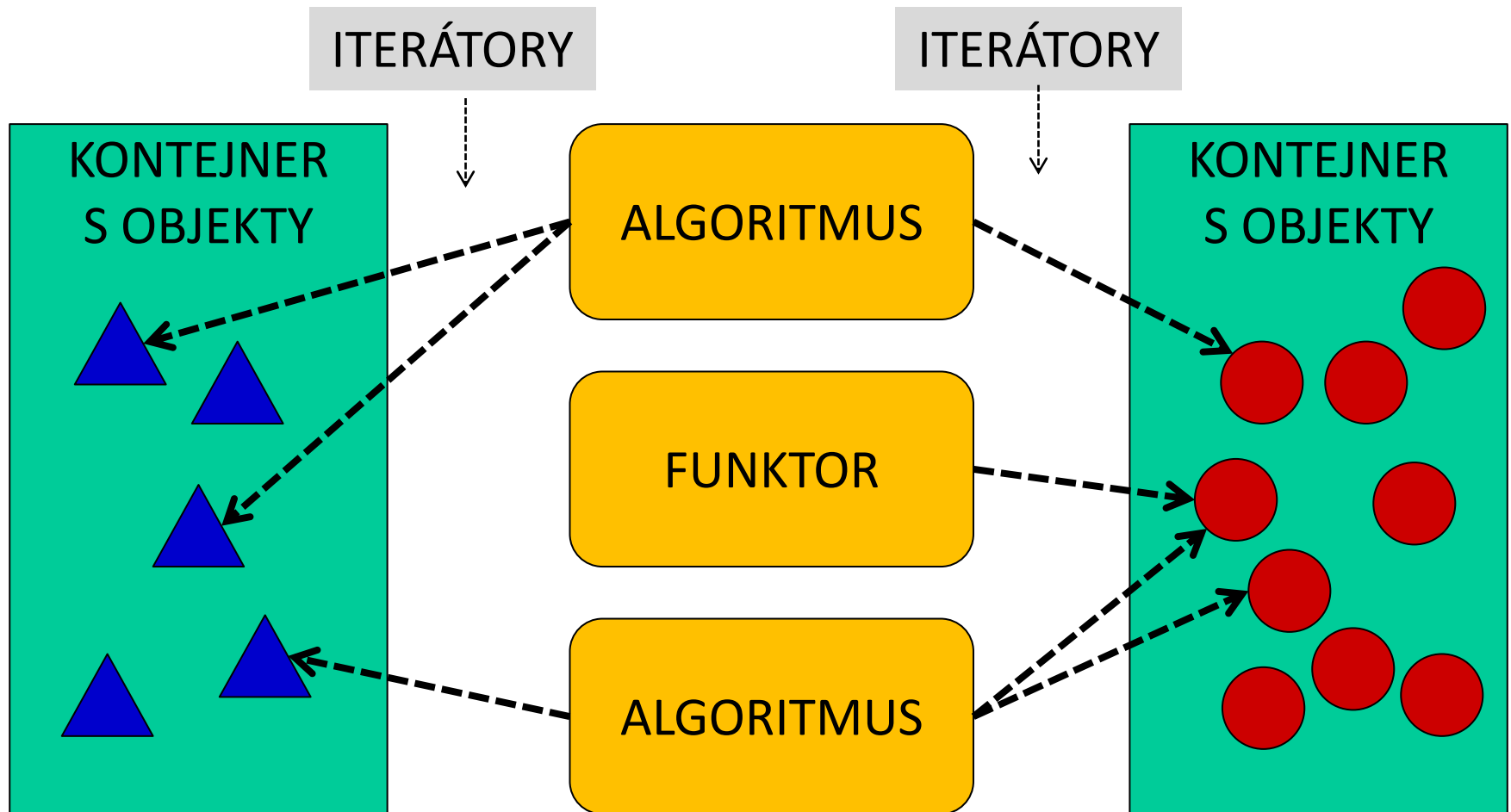
template <class K, class V>      //test existence prvku
bool SrtArrTab<K,V>::incl(const K& k) const {
    return search(k)>=0;
}

template <class K, class V>      //čtení hodnoty prvku
bool SrtArrTab<K,V>::read(const K& k, V& v) const {
    int j = search(k);
    if (j<0) return false;
    v = arr[j].val;              //předání hodnoty
    return true;
}
```

STL (Standard Template Library)

- Všeobecně dostupná implementace základních ADT
 - Součást C++ standard library
- Obsahuje generické třídy (šablony), umožňující vysokou flexibilitu datových typů
 - Podporuje generické programování
 - Odděluje algoritmy od ADT
 - Propojení prostřednictvím iterátorů
- Základní prvky
 - **Kontejnery**: objekty, obsahující jiné homogenní objekty
 - **Iterátory**: objekty s přetíženými operátory (např. **++**, **--**, **==**, **!=**, *****), analogie ukazatelů v kontejnerech
 - **Algoritmy** nad kontejnery
 - **Funktory**: volitelné funkcionální parametry algoritmů - objekty, pro které je přetížen operátor volání funkce **()**

Algoritmus, kontejner, funkce, iterátor



Algoritmy nebo parametrizovatelné/přetížené **funkce** (funkční objekty, funktory) využívají **iterátory** k interakci s objekty, uloženými v **kontejnerech**.

Kontejnery, obsažené v STL knihovně

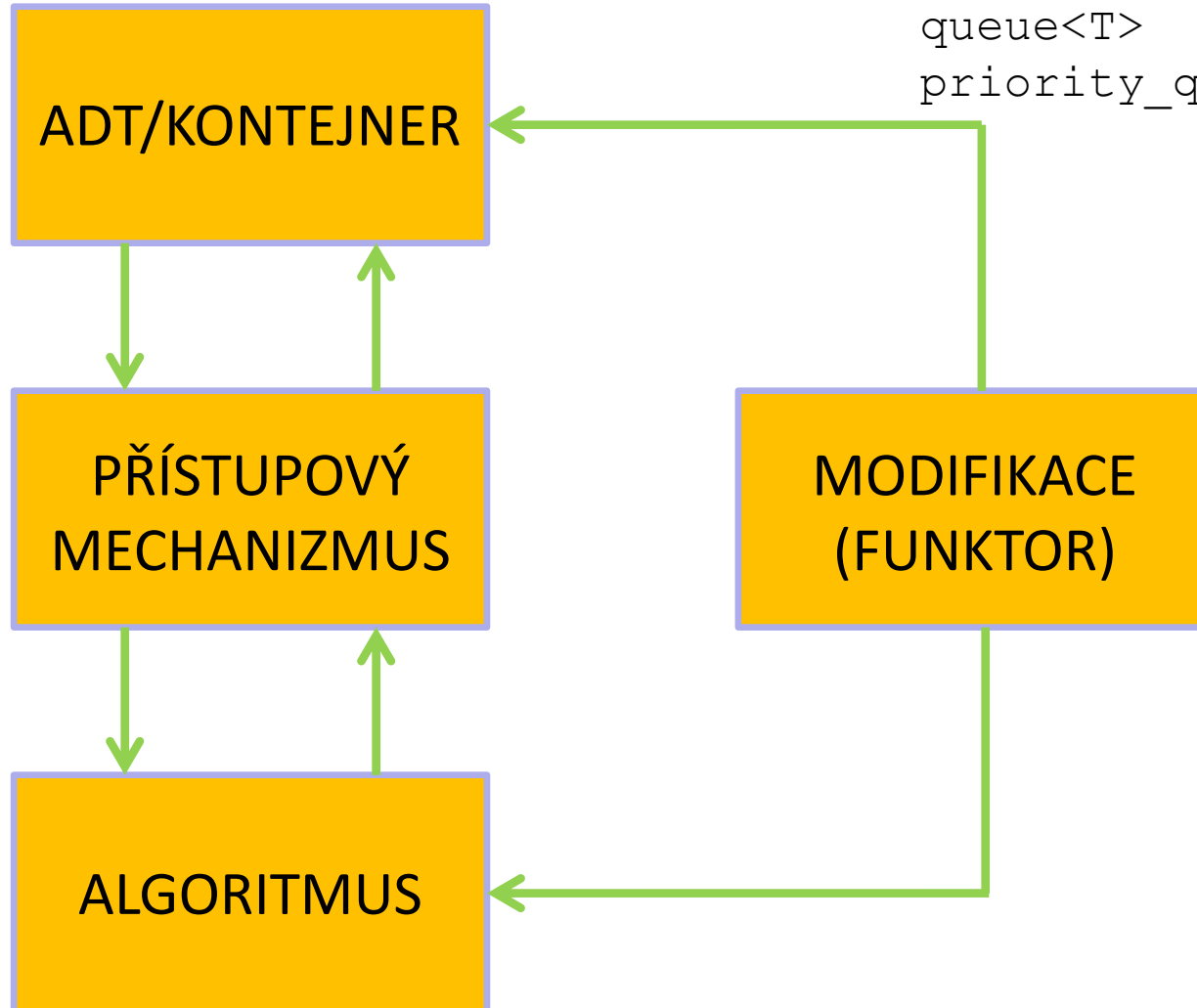
- Sekvenční kontejnery
 - array
 - vector
 - Deque („oboustranný *vector*“)
 - forward_list (optimalizace pro specifické typy dat - krátké seznamy, řídke matice)
 - list
- Kontejnerové adaptéry
 - stack (LIFO, obvykle využívá *deque*)
 - queue (FIFO, obvykle využívá *deque*)
 - priority_queue (řadí prvky, implementace *heap*, tj. kontejner *vector*)
- Asociativní kontejnery
 - set, unordered_set
 - multiset, unordered_multiset
 - map, unordered_map
 - multimap, unordered_multimap

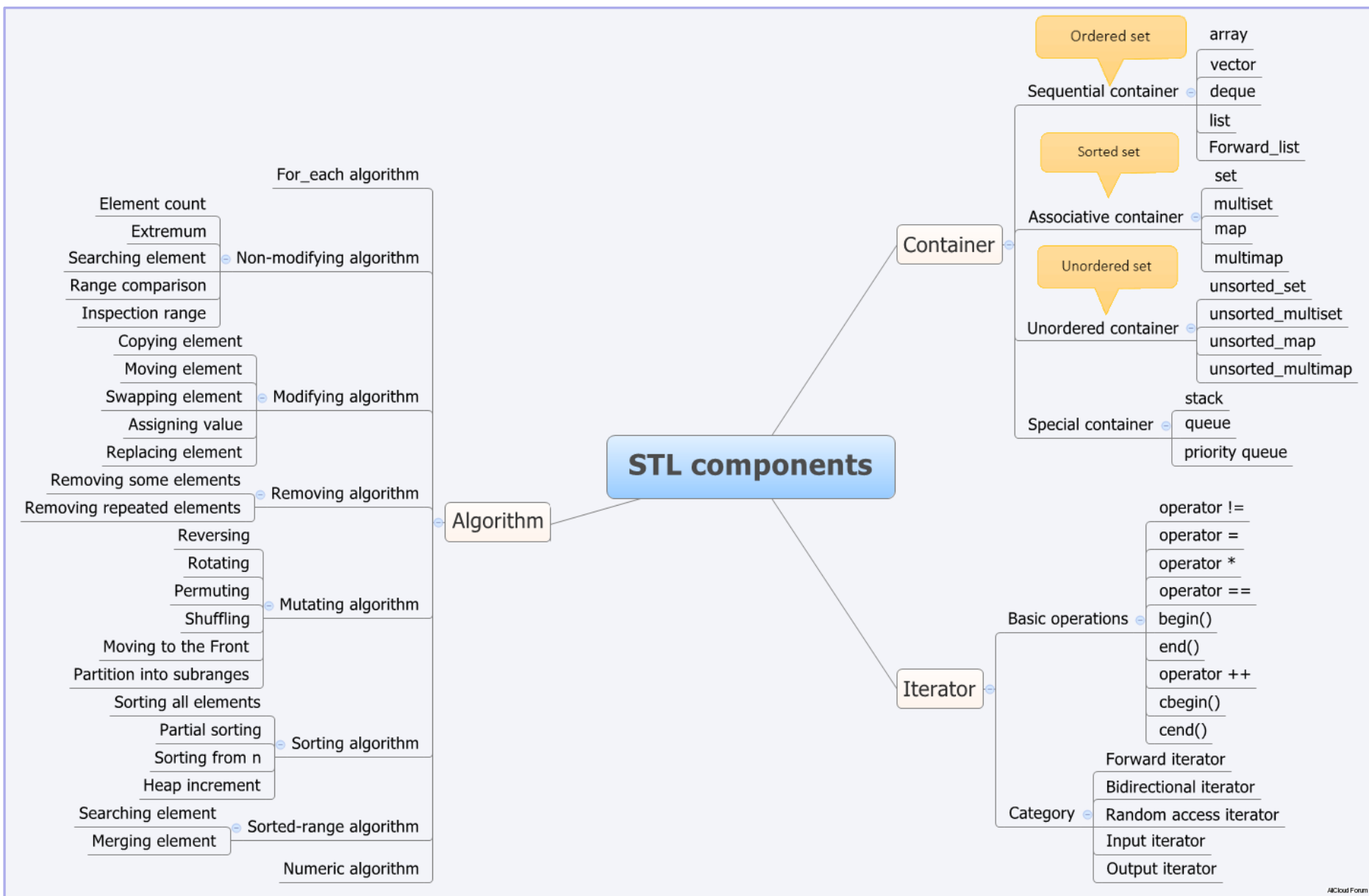
Filozofie práce s ADT v C++ (STL knihovna)

`vector<T>`
`list<T>`
`deque<T>`

`map<T>`
`set<T>`

`stack<T>`
`queue<T>`
`priority_queue<T>`

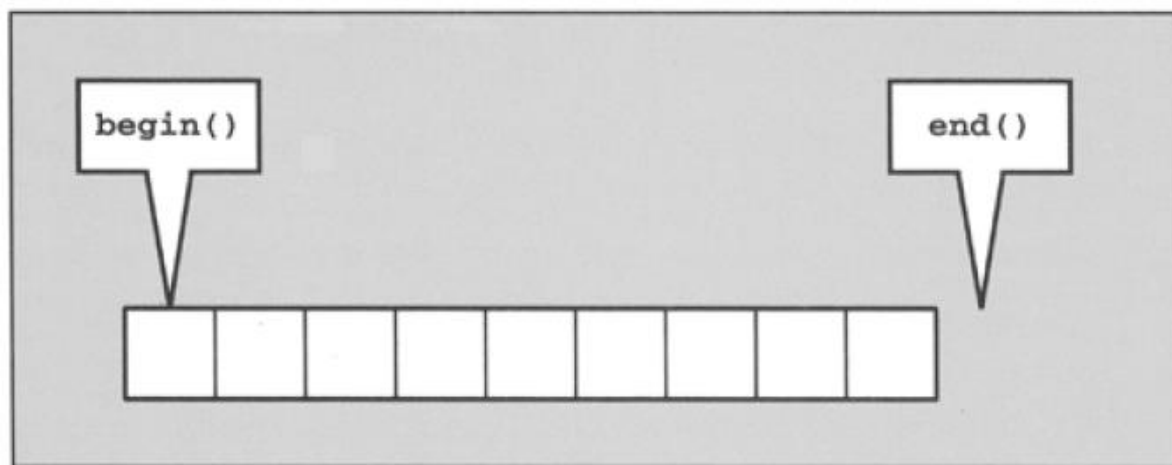




Iterátory

- Vstupní: `istream`
- Výstupní: `ostream`
- Dopředný
- Obousměrný: `list`, `set`, `m/set`, `map`, `m/map`
- Přímého přístupu (random): `vector`, `deque`, `array`

Základní funkce kontejnerů, vracející iterátory



Algoritmy

Nemodifikující:	<code>for_each, count, search ...</code>
Modifikující:	<code>copy, transform ...</code>
Odstraňující:	<code>remove, remove_if, ...</code>
Měnění:	<code>reverse, rotate, ...</code>
Řadicí:	<code>sort, partial_sort, ...</code>
Pro seřazená data:	<code>binary_search, merge ...</code>
Numerické:	<code>accumulate, partial_sum ...</code>

Standardní funktory (operátory)

- Srovnávací operátory
equal_to, not_equal_to, less, greater, less_equal, greater_equal
- Aritmetické operátory
plus, minus, multiplies, divides, modulus, negate
- Logické operátory
logical_not, logical_and, logical_or

Příklad k funktorům

- Funkce (konstanta 2 je obsahem funkce)
- Globální proměnná
- Šablona (hodnota *val* musí být známa v čase překladu)
- Funktor (parametrizovatelná funkce)
 - Řešení s největší flexibilitou

Funkce

```
void add2(int i) {  
    cout << i+2 << endl;  
}  
  
int main()  
{  
    vector<int> vec = { 2, 3, 4, 5};  
    for_each(vec.begin(), vec.end(), add2); // {4, 5, 6, 7}  
}
```

Globální proměnná

```
int val = 2;

void add2(int i) {
    cout << i+val << endl;
}

int main()
{
    vector<int> vec = { 2, 3, 4, 5 };
    for_each(vec.begin(), vec.end(), add2); // {4, 5, 6, 7}
}
```

Šablona

```
void add2(int i) {  
    cout << i+2 << endl;  
}  
  
template<int val>  
void addVal(int i) {  
    cout << val+i << endl;  
}  
  
int main()  
{  
    vector<int> vec = { 2, 3, 4, 5};  
    //for_each(vec.begin(), vec.end(), add2); // {4, 5, 6, 7}  
    for_each(vec.begin(), vec.end(), addVal<2>); // {4, 5, 6, 7}  
}
```

T

Funktor

```
class AddValue {
    int val;
public:
    AddValue(int j) : val(j) { }
    void operator()(int i) {
        cout << i+val << endl;
    }
};

int main()
{
    vector<int> vec = { 2, 3, 4, 5 };
    //for_each(vec.begin(), vec.end(), add2); // {4, 5, 6, 7}
    int x = 2;
    //for_each(vec.begin(), vec.end(), addVal<x>); // {4, 5, 6, 7}
    for_each(vec.begin(), vec.end(), AddValue(x)); // {4, 5, 6, 7}
}
```

Přístup k objektům pomocí indexů

```
using namespace std;
```

```
vector<int> IntVector;
```

```
IntVector.push_back(1);           //Přidání 3 prvků do vektoru  
IntVector.push_back(4);  
IntVector.push_back(8);
```

```
for(int y=0; y<IntVector.size(); y++)  
{  
    cout<<IntVector[y]<<" ";      //Výstup 1 4 8  
}
```

Přístup k objektům pomocí iterátoru

```
using namespace std;
```

```
vector<int> IntVector;
```

```
vector<int>::iterator IntVectorIterator;           //Iterátor
```

```
IntVector.push_back(1);
```

```
IntVector.push_back(4);
```

```
IntVector.push_back(8);
```

```
for(IntVectorIterator = IntVector.begin();
```

```
    IntVectorIterator != IntVector.end();
```

```
    IntVectorIterator++)
```

```
{
```

```
    cout<<*IntVectorIterator<<" ";           //Výstup 1 4 8
```

```
}
```

STL list, iniciace

```
#include <iostream>
#include <list>
using namespace std;

int main ()
{
    list<int> first;
    list<int> second(4,100); // 4 int
    list<int> third(second.begin(),second.end()); // iteračně
    list<int> fourth(third); // kopie
    int myints[] = {16,2,77,29}; //iter. z pole
    list<int> fifth(myints,myints+sizeof(myints)/sizeof(int));
}
```

STL list, příklad

```
#include <iostream>
#include <list>
using namespace std;

int main()
{
    list<int> seznam (10,7);
    list<int>::iterator it;
    for(it = seznam.begin(); it != seznam.end(); it++) {
        cout<<*it;
    }
    cout<<"\nHello World";
    return 0;
}
```

```
#include <cstdlib>           // využití EXIT SUCCESS
#include <iostream>          // I/O
#include <vector>             // vector
#include <algorithm>          // sort, random shuffle

using namespace std;        // eliminace std::
```

```
int main () {
    int a[ ] = {17, 12, 33, 15, 62, 45};
    vector<int> v(a, a + 6);           // inic. v jako kopie a
                                       // v: 17 12 33 15 62 45

    cout << v.size() << endl;         // Výstup: 6
    v.pop back();                     // v: 17 12 33 15 62
    cout << v.size() << endl;         // Výstup: 5
    v.push back(19);                  // v: 17 12 33 15 62 19
    cout << v.front() << " " << v.back() << endl;
                                       // Výstup: 17 19

    sort(v.begin(), v.begin() + 4);
                                       // v: (12 15 17 33) 62 19

    v.erase(v.end() - 4, v.end() - 2);
                                       // v: 12 15 62 19

    cout << v.size() << endl;         // Výstup: 4
}
```

```
char b[ ] = {'b', 'r', 'a', 'v', 'o'};
vector<char> w(b, b + 5);           // w: b r a v o
random_shuffle(w.begin(), w.end()); // w: o v r a b (např.)
w.insert(w.begin(), 's');          // w: s o v r a b

for (vector<char>::iterator p = w.begin(); p!=w.end(); ++p)
cout << *p << " ";               // outputs: s o v r a b
cout << endl;
return EXIT_SUCCESS;
}
```

STL mapa

```
map<string, int> myMap;           // a (string,int) map
map<string, int>::iterator p;     // an iterator to the map
myMap.insert(pair<string, int>("Rob",28)); // ins. ("Rob",28)
myMap["Joe"] = 38;                // insert("Joe",38), owld
myMap["Joe"] = 50;                // change to ("Joe",50)
myMap["Sue"] = 75;                // insert("Sue",75)
p = myMap.find("Joe");            // *p = ("Joe",50)
myMap.erase(p);                  // remove ("Joe",50)
myMap.erase("Sue");              // remove ("Sue",75)
p = myMap.find("Joe");
if (p == myMap.end()) cout << "nonexistent\n"; // output
for (p = myMap.begin(); p != myMap.end(); ++p) { // print all
    cout << "(" << p->first << "," << p->second << ")\n";
}
```

Příklad operací s mapou/slovníkem

<u><i>Operation</i></u>	<u><i>Output</i></u>	<u><i>Dictionary</i></u>
insert(5,A)	(5,A)	(5,A)
insert(7,B)	(7,B)	(5,A),(7,B)
insert(2,C)	(2,C)	(5,A),(7,B),(2,C)
insert(8,D)	(8,D)	(5,A),(7,B),(2,C),(8,D)
insert(2,E)	(2,E)	(5,A),(7,B),(2,C),(8,D),(2,E)
find(7)	(7,B)	(5,A),(7,B),(2,C),(8,D),(2,E)
find(4)	null	(5,A),(7,B),(2,C),(8,D),(2,E)
find(2)	(2,C)	(5,A),(7,B),(2,C),(8,D),(2,E)
findAll(2)	(2,C),(2,E)	(5,A),(7,B),(2,C),(8,D),(2,E)
size()	5	(5,A),(7,B),(2,C),(8,D),(2,E)
remove(find(5))	(5,A)	(7,B),(2,C),(8,D),(2,E)
find(5)	null	(7,B),(2,C),(8,D),(2,E)